

PHASE 2 TECHNICAL AUDIT &
CODEBASE REVIEW

Keja AI Platform Audit

A full-depth engineering review of the keja-ai repository: architecture, code quality, security, PWA, performance, SEO, and testing - with a consolidated findings register and a sequenced 90-day improvement roadmap.

Prepared by

Keja AI Engineering Audit Team

Repository [gadda00/keja-ai @ 86e2a66](#)

Date **September 2026**

47 FINDINGS · 28 CHAPTERS · 6 APPENDICES

Table of Contents

1. Executive Summary 9

 The platform in one paragraph 9

 Headline conclusions 10

 The ten findings that matter most 10

 What is working well 11

 What must change 12

 How to read this report 13

2. Engagement, Scope and Methodology 13

 Engagement context 13

 Scope 13

 Method 14

 Severity model 14

 Limitations 15

3. Product and Platform Overview 15

 What Keja AI is 15

 User journeys as built 16

 The honesty system 17

 User personas and jobs-to-be-done 17

 Delivery history 18

4. Market and Competitive Context 18

 Why a technical audit opens with the market 18

 The incumbent field 18

Demand-side realities	19
Supply-side dynamics	20
5. Architecture Assessment	20
The architectural pattern	20
Data flow	21
The hash router decision	22
Strengths of the architecture	22
Where the architecture has hit its ceiling	22
Assessment	23
6. Technology Stack Assessment	23
Stack inventory	23
The deadwood problem	24
Configuration posture	25
Version currency and risk	26
7. Source Inventory and Code Metrics	26
What is actually in the repository	26
Where the lines are	27
Size hotspots and what they mean	29
Repository hygiene	30
8. TypeScript and Code Quality	30
The paradox of the type system	30
Escape hatches in use	31
Patterns that carry the codebase	31
Smells and their severity	31

Git history as a quality signal	32
9. Component Architecture	32
The shape of the component tree	33
Monoliths	33
The shaden layer	33
Cross-cutting components worth studying	34
Assessment	34
10. State Management and Data Layer	35
The state architecture	35
Why this design works - and where it strains	36
The data dictionary discipline	36
The server-shaped gap	36
Assessment	37
11. AI Capability Assessment	38
What the AI actually is	38
The engine's architecture	38
The expectation gap and how to close it	39
Adjacent AI surfaces	39
12. Security Audit	40
Posture in one paragraph	40
Critical finding C-1: credential storage	40
Critical finding C-2: KYC data collection	41
High finding: the ungated admin console	41
High finding: no Content-Security-Policy	41

What the audit verified as clean	41
OWASP mapping	42
Threat model	43
Assessment	43
13. Data Model and Persistence	44
Two data models, one of them imaginary	44
The persistence reality	45
Migration gap analysis	45
Recommended schema shape	45
Assessment	46
14. Privacy and Regulatory Compliance	46
The privacy posture is a strength - until data gets real	46
Kenya Data Protection Act 2019 exposure	46
Registration and governance	47
Assessment	47
15. Progressive Web App Assessment	48
Installability verdict	48
The service worker	49
Gaps in an otherwise excellent implementation	49
iOS specifics	49
Assessment	50
16. Performance Assessment	50
Measured bundle composition	50
The recharts duplication	51

Imagery: the five-megabyte ghost	51
First-load waterfall	51
What is already fast	52
Assessment	53
17. SEO and Discoverability	53
The darkest chapter of the audit	53
The metadata that is done well	53
The sitemap that serves HTML	54
The strategic decision	54
Assessment	55
18. Accessibility and User Experience	55
Accessibility census	55
Contrast and motion	55
User experience inventory	56
The conversion gap	56
The missing map	57
The editorial and content system	57
Assessment	58
19. Mobile and Capacitor Strategy	58
A real strategy, not scaffolding	58
What ships and what is missing	58
Strategic value	59
Assessment	59
20. DevOps, CI/CD and Delivery	60

Pipeline inventory	60
What the pipeline gets right	60
Gaps in the delivery system	60
The Auto-Pilot as a delivery pattern	61
Workflow-by-workflow breakdown	61
Assessment	63
21. Observability	63
Production is a black box	63
Why the local analytics design helps the fix	64
Assessment and sequencing	64
22. Testing Assessment	64
Current state: zero	64
What existed, and what its loss means	65
Coverage gap analysis	65
Stack recommendation	66
Implementation sketch	66
Assessment	67
23. Consolidated Findings Register	68
How to read the register	68
Critical findings	68
High findings	69
Medium findings	70
Low findings	72
24. Phase 2 Target Architecture	72

The one-seam principle	72
The API seam	73
The data layer	74
Authentication	74
Payments: M-Pesa done correctly	74
The SEO decision, implemented	75
API contract sketch	75
Sequencing	76
25. Phase 2 Data Schema Blueprint	76
From dictionary to schema	76
Enums and integrity rules	78
Migration waves	78
26. Engineering Excellence Plan	79
Guardrails back on	79
The purge	79
Refactoring with intent	79
Craft standards to adopt	80
What deliberately stays	80
27. 90-Day Action Plan	80
Sequencing logic	80
Effort and impact matrix	83
Cadence and verification	84
28. Risk Register	84
How the register was built	84

Closing statement 85

The register's shape 86

Appendix A: Route Inventory 86

Appendix B: localStorage Registry 88

Appendix C: Dependency Audit 90

Appendix D: Document and Asset Inventory 91

Appendix E: Glossary 93

Appendix F: Methodology Notes and Reproduction 95

Reproducing the audit 95

Tooling and standards referenced 95

Confidence and freshness 96

PART I - EXECUTIVE SUMMARY AND ENGAGEMENT CONTEXT

1. Executive Summary

This report is the Phase 2 deliverable for the Keja AI platform: a full-depth technical audit of the **gadda00/keja-ai** repository, conducted from the perspective of a senior, multi-disciplinary engineering team. Phase 1 (August-September 2026) took the platform from concept to a production deployment on Vercel serving **keja.app**, with a working PWA, a CI/CD pipeline, and a self-growing listing inventory. Phase 2 is the moment to slow down, read every corner of the codebase honestly, and decide what must be true of this platform before it can accept real money, real identity documents, and real landlords. This document is that read, in full.

The platform in one paragraph

Keja AI is a rental-property marketplace for Kenya, named after the Swahili word for home. It is implemented as a Next.js 16 static-export single-page application: one HTML shell, 52 hashed asset chunks, a custom hash router driving 32 views, and a complete absence of any server-side runtime. All state - authentication, listings, landlord ledgers, tenant applications, a tokenization sandbox with KYC forms - persists in browser `localStorage` under namespaced keys. The deployment pipeline is genuinely strong for a project of this maturity: GitHub Actions gates every push with typecheck, lint, and a static build; Vercel serves the artifact with hardened security headers; an hourly monitor verifies liveness; and a cron-driven Auto-Pilot pipeline grows the listing inventory four times a day through audited, quality-gated commits. The codebase is 33,253 lines of TypeScript across 136 source files, supported by 435 tracked files including full Android and iOS Capacitor shells and an unusually complete documentation suite.

435 files tracked in git	33,253 lines of TS/TSX in src/	32 hash-routed views	0 server-side endpoints
------------------------------------	--	--------------------------------	-----------------------------------

Headline conclusions

The audit team reached five conclusions with high confidence. First, **the platform is an excellent demo and a credible seed, but it is not yet a product**: every feature that implies a transaction - bookings, messages, payments, KYC, escrow - is a browser-local simulation that never leaves the device. Second, **the engineering discipline visible in the code is well above average for this stage**: zero uses of *any* and *@ts-ignore* across the entire source tree, a coherent documented store architecture, honest data-labelling conventions, and strong accessibility hygiene. Third, **the platform's largest technical debts are invisible to users but decisive for the business**: hash routing makes all 87 property pages invisible to search engines, the production build ships with no sitemap despite robots.txt declaring one, and there is no error tracking or product analytics in production at all. Fourth, **the testing story regressed catastrophically during the Next.js rebuild**: 23 test suites totalling 3,039 lines existed in early September and were deleted in the migration commit, leaving the platform with zero automated tests today. Fifth, **two security exposures become critical the moment real users arrive**: passwords are stored with a reversible, unsalted DJB2 hash in localStorage, and the KYC form collects national ID numbers and source-of-funds declarations into the same plaintext store - a combination that would fall foul of the Kenya Data Protection Act 2019 if exercised with real data.

The core strategic judgment of this audit: Keja AI's static-first architecture was the right call for Phase 1 - it produced an offline-capable, installable, cheap-to-run platform in weeks. The same architecture is the wrong call for Phase 2. The platform now needs one well-defined seam - a typed API layer in front of PostgreSQL, Prisma, and M-Pesa - while keeping the entire existing client surface. The store abstraction already in the codebase makes this migration incremental rather than a rewrite.

The ten findings that matter most

Rank	Severity	Finding	Primary chapter
1	Critical	Passwords stored with reversible DJB2 hash; open registration accepts real credentials	Ch. 12 Security
2	Critical	KYC collects national ID and source-of-funds into plaintext client storage	Ch. 12, 12
3	Critical	All 87 listing pages, articles, and guides invisible to search engines	Ch. 17 SEO
4	Critical	Zero automated tests; the pre-rebuild safety net was deleted in the migration	Ch. 22 Testing
5	High	Admin console renders without any authentication check	Ch. 12 Security
6	High	No Content-Security-Policy on an otherwise well-hardened edge	Ch. 12 Security
7	High	sitemap.xml missing while robots.txt declares it; serves HTML at 200	Ch. 17 SEO
8	High	No production error tracking or analytics of any kind	Ch. 21 Observability
9	High	recharts shipped twice (~300 KB) plus ~5 MB dead imagery on the CDN	Ch. 16 Performance
10	High	Primary conversion CTAs are toast stubs; only WhatsApp is real	Ch. 18 UX

Table 1: The audit's top ten findings, ranked by consequence (full register: Chapter 23).

What is working well

It is worth stating clearly, before the findings register fills the next hundred pages, how much of this repository is genuinely good engineering. The service worker implementation is in the top tier of what the audit team sees in production PWAs: content-hash versioning stamped into the file at build time, a three-tier caching strategy with an eight-second network timeout, a real offline fallback page, and seventeen iOS launch screens covering every device class. The accessibility work - 393 ARIA attributes, a focus-trap utility with stacked-dialog awareness, a skip link, labelled forms, and consistent icon semantics - exceeds what most commercial marketplaces ship. The honesty system (a machine-readable claims register, FACT/ESTIMATE/ASSUMPTION labelling in the AI engine, trust ceilings that cap synthetic listings below verified ones) is a genuine differentiator in a market where listing fraud is the primary user complaint. And the operational documentation - deployment runbooks, a domain migration forensic record, a data dictionary, twenty partner proposals - would be the envy of teams ten times this size.

What must change

The audit registers 47 findings across seven domains: 4 Critical, 13 High, 22 Medium, and 8 Low. The Critical findings cluster around credential storage, KYC data exposure, search invisibility, and the absence of any regression safety net. The High findings include an ungated administration console, a missing Content-Security-Policy, a sitemap that returns HTML with a 200 status, zero production observability, duplicated charting code shipping roughly 300 KB of redundant JavaScript, and five megabytes of dead WebP imagery deployed to the CDN. None of these are difficult fixes in isolation; the risk is compounding. A platform that cannot see its own errors, cannot be found by search, and cannot verify that a refactor did not break the mortgage calculator is a platform whose iteration speed is borrowed against future incident cost.

Domain	Verdict	Score	Defining evidence
Architecture	Coherent, honest, and at its ceiling; needs the Phase 2 seam	3.5	Static export + hash router + localStorage; mini-services empty
Code quality	Clean surface, weakened gates, heavy deadwood	3.0	0 any/@ts-ignore; 26 lint rules disabled; 4,243 LOC dead UI
Security	No server attack surface; client-side trust is decorative	2.0	DJB2 password hash; #/admin ungated; no CSP
Data and persistence	Well-modelled locally; no persistence layer exists	2.0	Prisma scaffold untouched; 28 localStorage keys
PWA and mobile	Top-tier implementation	4.5	SW versioning; 17 iOS splashes; Capacitor shells
Performance	Acceptable today, wasteful under the hood	3.0	611 KB initial JS; recharts shipped twice
SEO	Effectively dark to search engines	1.5	Hash routing; sitemap 404-as-HTML; no JSON-LD
Testing	Zero automated tests today	1.0	3,039 LOC of suites deleted in rebuild commit
DevOps and CI/CD	Strong gates, no previews, blind in production	3.5	tsc+lint+build gates; hourly monitor; no Sentry

Table 2: Top-level audit scorecard by domain (1 = weak, 5 = strong).

How to read this report

The report is organised in six parts. Part I (this summary, the engagement context, and the product overview) frames what was audited and how. Part II assesses the system as built: architecture, technology stack, and source inventory. Part III reads the code closely: TypeScript quality, component architecture, and the state layer. Part IV covers security, data modelling, and regulatory exposure under the Kenya Data Protection Act. Part V evaluates every user-facing and operational discipline: PWA, performance, SEO, accessibility, mobile, DevOps, observability, and testing. Part VI consolidates the findings register and delivers the recommendations: a Phase 2 target architecture, an engineering excellence plan, a sequenced 90-day action plan, and a risk register. Five appendices provide the raw inventories - routes, storage keys, dependencies, documents, and methodology - so that every claim in the body of this report can be traced to something countable in the repository.

2. Engagement, Scope and Methodology

Engagement context

Keja AI reached its Phase 1 conclusion on 10 September 2026, with commit 86e2a66 promoted to production on Vercel under the keja.app domain, the Netlify infrastructure retired, and all four GitHub Actions workflows green. The platform owner then commissioned this Phase 2 engagement with a deliberately broad mandate: return to the repository as a highly skilled team of professionals would, analyse everything, and produce a detailed technical report - including deeply researched recommendations and improvements - that could serve as the engineering agenda for the next phase of the product. This document is that report. It is written to be actionable: every finding cites files and line ranges, every recommendation is sequenced and sized, and the closing chapters provide a 90-day plan that a small team can execute without further discovery.

Scope

The audit covers the entire git-tracked surface of the repository at commit 86e2a66: 435 files spanning the Next.js application source, the build and deployment configuration, the GitHub Actions workflows, the Prisma schema, the Capacitor Android and iOS projects, the public assets including the service worker and manifest, the operational scripts including the Auto-Pilot listing pipeline, and the documentation suite. The production build artifact (the 21 MB static export) was sampled to measure real bundle composition. Git history - all 106 commits - was read to reconstruct decisions and identify regressions, most significantly the loss of the pre-rebuild test suite. Out of scope: load testing, penetration testing, user research, and financial modelling of the business itself. The audit examined the code as deployed, not merely as written; where the two differ (for example, the sitemap generator that exists as a script but never runs in CI), the deployed behaviour is treated as the truth.

Method

The audit was executed as four parallel workstreams, mirroring how a professional platform team would divide a review: architecture and technology stack; code quality and implementation; security, data model, and backend; and frontend experience, PWA, performance, SEO, and DevOps. Each workstream combined automated measurement (git inventory, line counts, bundle analysis, targeted greps for vulnerability patterns such as `dangerouslySetInnerHTML`, hardcoded secrets, and disabled compiler flags) with close reading of the significant files - the 1,049-line conversational engine, the 556-line authentication layer, the service worker, the deploy workflow, and the Prisma schema among them. Findings were then cross-verified between workstreams, consolidated into a single register with severities, and mapped to a remediation roadmap. External research - current Next.js rendering guidance, Safaricom Daraja integration requirements, Kenya Data Protection Act registration obligations, and 2026 testing-stack conventions - grounds the recommendations in present-day practice rather than audit-team preference.

Workstream	Techniques applied	Primary outputs
A - Architecture and stack	git ls-files inventory; config forensics (next.config.ts, vercel.json, tsconfig, eslint); route table extraction; dependency usage analysis	Route inventory; dead dependency list; architecture assessment; repo hygiene findings
B - Code quality and testing	LOC metrics; pattern greps (any, ts-ignore, non-null assertions, console, TODO); component import graph; git archaeology of the deleted test suites; lint configuration review	Quality findings with file:line citations; testing gap analysis; dead component register
C - Security and data	Full Prisma schema read; secrets sweep across all 106 commits; OWASP 2021 mapping; localStorage key enumeration; XSS vector grep; workflow script-injection review	Findings table with OWASP categories; data model gap analysis; Kenya DPA exposure points
D - Frontend and operations	Bundle measurement from out/; PWA checklist against current installability criteria; SEO surface inspection (metadata, sitemap, JSON-LD); all y attribute census; workflow and header analysis	Bundle tables; PWA pass/fail matrix; SEO and observability findings; DevOps maturity notes

Table 3: Audit workstreams, techniques, and primary outputs.

Severity model

Findings are rated on a four-point scale anchored to business consequence rather than code aesthetics. **Critical** findings either expose real users to concrete harm if the platform scales as intended, or block a primary business channel outright. **High** findings materially degrade security, growth, velocity, or reliability, and will predictably become incidents if unaddressed. **Medium** findings waste effort, budget, or user patience, and compound quietly. **Low** findings are hygiene: they matter to craftspeople and to anyone who inherits this code, but they will not decide the fate of the platform. Severity is assigned to the finding as it stands today, not as it might become; where a latent issue turns critical only upon a specific trigger (for example, the first real user entering a real password), the register says so explicitly.

Limitations

Three limitations bound this audit. First, it is a static analysis: no dynamic testing, penetration testing, or user observation was performed, and runtime behaviours are inferred from code and build artifacts rather than instrumented sessions. Second, the audit deliberately did not modify the repository; every recommendation in Part VI remains to be implemented, and effort estimates are therefore planning-grade rather than committed. Third, market and regulatory judgements - competitor positioning, CMA sandbox prospects, ODPC enforcement risk - rely on published sources current as of September 2026 and should be re-validated with local counsel before being relied upon in investment or launch decisions. None of these limitations materially softens the code-level findings, which rest on direct evidence in the repository.

3. Product and Platform Overview

What Keja AI is

Keja AI positions itself as an AI-assisted rental platform for Kenya - a marketplace where tenants find verified homes, landlords manage units and rent, and investors analyse yields, with a WhatsApp-first communication culture and an explicit design bias toward the constraints of the Kenyan market: intermittent connectivity, Android-dominant devices, M-Pesa as the payment rail, and deep-seated distrust of listing fraud. The product umbrella is broad for a platform this young: the marketing surface describes nine products across four portals, from the core property search through a deal-analyst document screener, an investment calculator and portfolio dashboard, a tenant hub, a diaspora hub, a developer portal, an institutional portal, a valuation desk, and a tokenization sandbox aimed eventually at the CMA regulatory sandbox. This breadth is strategically deliberate - the STRATEGY.md document argues a Kenya-first, trust-first wedge - but it means the audit must repeatedly distinguish between surfaces that are real, surfaces that are honest simulations, and surfaces that are aspiration wearing a UI.

Product surface	Route	Implementation reality	Server dependency required next
Property search	#/properties	Real: 87 merged listings, facets, sorting, compare, saved searches with local alert matching	None (data freshness only)
Listing detail	#/properties/:id	Real: gallery, trust breakdown, passport, inline mortgage calculator	Lead capture form
Ask Keja (AI advisor)	#/ask	Offline rule-based engine, 1,049 LOC, EN/SW/FR, escalation guard to WhatsApp	Optional LLM upgrade path
Deal Analyst	#/deal-analyst	On-device document pre-screening heuristics	Server-side parsing and persistence
Investor tools	#/invest, #/portfolio	Full calculators, portfolio dashboard, recharts visualisations - all client-side	Real market data feeds
Landlord management	#/manage	Units, tenants, payment ledger, maintenance tickets - localStorage only	Multi-device sync, M-Pesa reconciliation
Tenant hub	#/tenant	Applications with income/employer data - localStorage only	Submission API, landlord notification
Tokenization trial	#/tokenize	Trial wallet (USD 25,000 credits), KYC form, order book simulation	CMA sandbox, custody, real ledger
Finance / transact	#/finance, #/transact	Mortgage and affordability calculators; transaction flow is presentational	PSP integration, escrow model

Table 4: The nine-product surface, classified by implementation reality as of commit 86e2a66.

User journeys as built

The primary tenant journey is fully functional as a client-side experience: search by area, type, price, and trust score; open a listing with a verification evidence panel and freshness chips; save favourites and comparisons; then either start a WhatsApp conversation with a prefilled listing reference or hit the book-a-viewing button - which, today, shows a toast and nothing else. The landlord journey mirrors property-management software: register units, record tenants and rent payments against an M-Pesa/cash/bank channel ledger, raise maintenance tickets, and read dashboard metrics computed on-device. The investor journey provides yield mathematics honest enough to be useful - correct amortisation, named Kenyan cost constants, a KES/USD peg - with every number derived from the seeded catalog rather than live market data. The critical structural fact across all journeys is that **no data crosses devices**: a landlord on one phone sees nothing a tenant did on another. This is the single largest functional gap between the product narrative and the shipped reality, and it defines the Phase 2 agenda.

The honesty system

The platform's most distinctive engineering artefact is its honesty infrastructure, and the audit wants to name it explicitly because it materially raises the platform's trust ceiling. The claims register (`src/data/claims.ts`) is a machine-readable inventory of what the platform asserts about itself, consumed by a public Trust Center view. The AI engine labels every material statement as FACT, ESTIMATE, ASSUMPTION, or REPORTED, with documented thresholds. Auto-Pilot listings are capped below verified listings in trust scoring and are forbidden from claiming Ardhisasa verification. Demo surfaces carry a persistent banner. Verification records mark their own method as simulated. In a market where the incumbent portals' biggest weakness is phantom listings, this discipline is not decoration - it is the product's most defensible differentiator, and every Phase 2 recommendation in this report is constrained to preserve it.

User personas and jobs-to-be-done

The platform's surfaces map onto five personas the audit inferred from the product decisions (and which the strategy documents corroborate). Rendering them explicit matters for the recommendations that follow, because each persona is exposed to a different subset of the findings register - the renter meets the conversion theatre and SEO darkness; the landlord meets the single-device ceiling; the diaspora investor meets the KYC exposure; the agent meets the lead-capture gap; and the regulator meets the claims register, for better and occasionally for worse.

Persona	Primary job-to-be-done	Current experience	Gating findings
Urban renter	Find a trustworthy home fast, within budget, in a known area	Search + trust scores strong; WhatsApp handoff real; viewing booking theatrical	F-03 SEO, F-26 CTAs, F-12 no map
Landlord	Fill units, collect rent, track arrears without spreadsheets	Full console that works on one device only	Ch. 10 ceiling, F-05 admin gate, P2-4 payments
Diaspora investor	Buy and oversee Kenyan property remotely with verified evidence	Evidence panels + investor tools; tokenization trial simulated	F-02 KYC exposure, R8 CMA sequencing
Agent	Receive qualified leads, convert in chat	WhatsApp deep links with listing references; no lead form	F-26, F-09 share unfurls
Regulator / partner	Assess whether the platform's claims are true	Claims register is best-in-class; test-claim citations stale	F-04 doc currency, Ch. 14 posture

Table 5: Personas, their jobs-to-be-done, and the findings that gate them.

Delivery history

The repository compresses an unusual amount of history into 13 days: 106 commits between 28 August and 10 September 2026, on a single main branch, from eight author identities reflecting a blend of human and automated work. The defining event is commit ae0b2e3, the Next.js 16 rebuild: 254 files changed, 19,783 lines added and 40,887 deleted, replacing an earlier Vite SPA wholesale - and, with it, deleting 23 test suites and the entire quality tooling layer (vitest configuration, coverage scripts, husky hooks, prettier) that the previous iteration had assembled. Since that commit, 38 more commits have shipped, 34 of them Auto-Pilot listing ingestions. The velocity is impressive; the deleted safety net is the subject of Chapter 22. What follows in this report is a portrait of a codebase that is simultaneously young, well-tended in places, and carrying a small number of very load-bearing omissions.

4. Market and Competitive Context

Why a technical audit opens with the market

Engineering priorities for a marketplace cannot be set in a vacuum: the disciplines this report scores - SEO, PWA installability, M-Pesa readiness, trust infrastructure - are precisely the axes on which the Kenyan property-technology market is currently competing. This chapter therefore fixes the competitive frame that the rest of the report's recommendations assume, so that when Chapter 24 ranks prerendering above refactoring or payments behind authentication, the ranking is legible as a market judgment and not merely an engineering preference.

The incumbent field

The Kenyan rental discovery market is dominated by three classes of player. **BuyRentKenya**, backed by ROAM Group (Ringier One Africa Media), is the most trafficked dedicated property portal in East Africa and the direct comparator for Keja's browse-and-enquire flow - its strengths are inventory depth and years of SEO authority; its documented weaknesses are the classic portal ills: stale and phantom listings, unverified agents, and a lead-capture experience that resells enquiries. **Jiji** operates the horizontal-classifieds model at enormous scale, with pricing power and aggressive distribution but minimal property-specific structure - no trust scoring, no verification evidence, no financing surfaces. **Property24** and the agency-owned portals occupy the premium-listing tier. Around the portals sits a fast-professionalising **property-management software** segment - local platforms marketing M-Pesa-integrated rent collection, invoicing automation, and increasingly AI-flavoured tenant screening - which is exactly the landlord-side surface Keja's #/manage console sketches. The structural gap in the field is the one Keja's strategy documents already target: nobody owns verified trust, and nobody owns the tenant-landlord transaction end to end with local payment rails.

Capability	BuyRentKenya	Jiji	PM software segment	Keja AI (shipped)
Inventory depth	Deep	Deepest	n/a (B2B)	87 listings (60 synthetic)
Listing verification	Partial	None	n/a	Designed, simulated (F-02 adjacent)
Trust scoring / evidence	None public	None	n/a	Strongest in class (offline)
SEO authority	High	High	Low-mid	Zero (Critical F-03)
PWA / installable app	Store apps	Store apps	Varies	Top-tier PWA, shells dormant
M-Pesa integration	Lead-gen only	Lead-gen only	Core product	Modelled, not integrated
AI surfaces	Marketing claim	None	Emerging (screening)	Real rule-based engine, offline
Cost to tenant	Free + agent fees	Free	SaaS for landlords	Free

Table 6: Capability positioning: Keja AI (as shipped) versus the incumbent field (audit synthesis, 2026).

Demand-side realities

Three demand-side realities shape what Phase 2 should optimise. First, search remains the primary discovery channel for serious renters - portal-industry practice and the platform's own marketing analysis agree that high-intent users arrive via search queries for areas, budgets, and configurations, which is what makes the SEO finding existential rather than cosmetic. Second, WhatsApp is the transaction rail of trust: Kenyans enquire, negotiate, and follow through in chat, which is why the platform's WhatsApp-first handoff is a genuine market fit rather than a stopgap - provided the CTAs never pretend to be more than they are (finding F-26). Third, mobile data economics reward the offline-capable PWA: a listing app that re-serves its shell and imagery from cache respects budgets that incumbent portals' ad-laden pages do not - an installability story that only pays while the installed experience stays flawless (Chapter 15's gaps matter here).

Supply-side dynamics

On the landlord side, the professionalising segment signals where willingness-to-pay sits: rent collection with M-Pesa reconciliation, arrears visibility, and maintenance ticketing - the exact feature set of the `#/manage` console, currently `localStorage`-only. The diaspora-investor segment, which Keja explicitly courts with a dedicated hub and tokenization ambitions, values verified representation from afar - the trust register and evidence panels are the right instruments, and the KYC exposure (finding F-02) is the regulatory toll that segment charges. The audit's market read, consolidated: the platform's differentiation (honesty infrastructure, offline PWA, AI-laboured assistance) is real and rare; its deficits (inventory, SEO, transactions) are exactly the ones incumbents cannot quickly copy culture around - and exactly the ones the Phase 2 roadmap in Part VI sequences.

PART II - SYSTEM ASSESSMENT

5. Architecture Assessment

The architectural pattern

Keja AI is a **static-export single-page application with a build-time data pipeline**. The Next.js build produces a directory of static assets (`out/`, 21 MB) with one HTML shell; a custom hash router (`src/lib/router.tsx`) implements navigation over hashchange events; and every view, store, and AI routine executes in the browser. There are no API routes, no server actions, no middleware, and no serverless functions: the mini-services directory contains only a `.gitkeep` placeholder. Vercel's configuration confirms the intent - framework is null, every path rewrites to `index.html`, and the build command stamps the service worker before deploy. This is a deliberate architecture, documented in the repository's own engineering dossier, and it was chosen for reasons that hold up well: it yields an offline-capable PWA, an installable app on both mobile platforms from a single bundle, near-zero hosting cost, and a trivially auditable security surface (no server to attack).

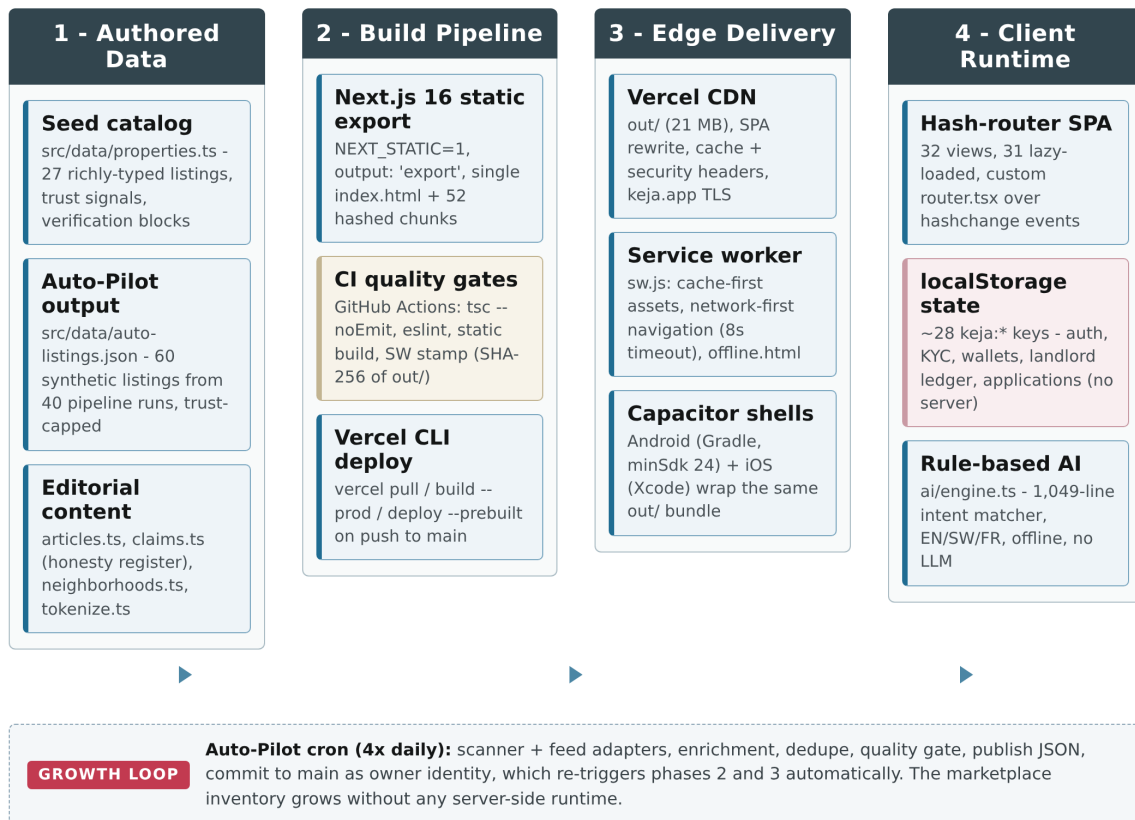


Figure 1: Current system architecture: four phases from authored data to client runtime, plus the Auto-Pilot growth loop.

Data flow

Data enters the system at build time through two authored sources: the seed catalog (src/data/properties.ts, 27 richly-typed listings with verification blocks, trust signals, and yield data) and the Auto-Pilot output (src/data/auto-listings.json, 5,630 lines carrying 60 synthetic listings produced by 40 quality-gated pipeline runs). Editorial content - articles, area guides, the claims register, tokenization assets - is authored in TypeScript alongside. At runtime, an inventory layer (src/lib/inventory.ts) merges these sources with admin-approved user submissions from localStorage into a single collection, applying the trust rules that cap synthetic listings below verified ones. From there, nine domain stores own their slices of user state. The growth loop closes the circle: a GitHub Actions cron job runs the Auto-Pilot scanner four times daily, and when it finds new listings, it commits them to main, which re-triggers the build and deploy. The marketplace literally grows through its own CI.

The hash router decision

The custom hash router deserves its own treatment because it is the highest-leverage architectural decision in the repository. Hash routing (`#/properties/KJA-001`) works offline, requires no server rewrite rules, behaves identically inside Capacitor shells and on the open web, and never produces a 404. Those are real virtues for the Phase 1 constraints. The cost is equally real and lands on the business rather than the code: search engines discard URL fragments, so every property page, every article, and every area guide is invisible to organic search; social shares unfurl as the generic home page because there is no per-route metadata; and the sitemap - even if it were generated - would point at fragment URLs that Google will not index as distinct pages. For a listings marketplace whose primary competitors (BuyRentKenya, the most-trafficked dedicated property portal in East Africa, and Jiji with its enormous classifieds footprint) acquire users predominantly through search, this is an existential channel decision, not a technical preference. Chapter 24 frames the two honest options: accept a PWA-first, WhatsApp-distributed product, or reintroduce path-based routes with prerendering while keeping the SPA shell.

Strengths of the architecture

- **Single-concern clarity.** Everything the client needs ships in one immutable artifact; there is no API versioning, no partial deployment, no server state to reason about.
- **Offline and install parity.** The same bundle serves `keja.app` over the CDN and the Android/iOS Capacitor shells from disk, so the PWA and native tracks cannot drift.
- **Auditable security surface.** With no endpoints, the OWASP server-side categories simply do not apply; the audit confirmed zero network egress from the client bundle.
- **CI-native data operations.** Listing changes arrive through reviewed, attributable commits with quality gates - arguably safer than an admin UI backed by a database at this team size.
- **Honest boundaries.** Module headers state the `localStorage` ceiling (5 MB, single-device, client-mutable) rather than hiding it; the demo surfaces label themselves.

Where the architecture has hit its ceiling

Three ceilings are now load-bearing. The first is **persistence**: `localStorage` cannot cross devices, cannot exceed roughly five megabytes, cannot be backed up, and is trivially mutable by any user - which is fine for favourites and fatal for rent ledgers, KYC records, and authentication. The second is **truth**: with no server, there is no shared fact between a landlord and a tenant, which is the definition of a marketplace. The third is **trust in computation**: financial calculations and authorization decisions execute in the client, where they can be inspected and altered; a payments-relevant platform eventually needs its arithmetic and its permissions to live somewhere a user cannot edit. None of these ceilings argues for abandoning the SPA - they argue for the single seam described in Chapter 22, behind which PostgreSQL, real authentication, and M-Pesa can live while the client surface stays recognisably itself.

Force	Static-first wins	Static-first costs
Offline capability	Full offline shell with cached listings; SW three-tier strategy	Conflicts with future live-inventory freshness
Hosting economics	CDN-only cost near zero on Hobby tier	Any backend addition reintroduces server budgeting
SEO / acquisition	No infrastructure to scale; deploys are atomic	Fragment URLs unindexable; channel effectively closed (see Ch. 17)
Security surface	Nothing to attack server-side; no secrets in bundle	All trust decisions client-side and mutable; no CSP yet
Data integrity	Nothing to corrupt centrally; per-device isolation	No shared truth; ledgers and KYC are suggestions, not records
Delivery speed	One artifact, one pipeline, no environments	No preview environments possible on current Hobby setup

Table 7: Architectural forces: what the static-first design wins and what it costs.

Assessment

Scored as a Phase 1 architecture, this design earns strong marks: it is coherent, documented, honest about its limits, and it shipped. Scored as a foundation for the business the product narrative describes - verified rentals, managed payments, tokenized assets - it is a seed stage. The audit's judgment is that the architecture should be **extended, not replaced**: the store abstraction (one generic useStore primitive feeding nine domain hooks) is precisely the interface a server-backed persistence layer needs, and the route layer's isolation behind a custom router means the router itself can evolve without touching views. The worst outcome would be a rewrite; the second worst would be pretending the ceilings are further away than they are. The recommendations in Part VI are sequenced to avoid both.

6. Technology Stack Assessment

Stack inventory

The declared stack is modern and, at its core, correctly chosen. Next.js 16.1.3 (locked) on React 19.2.3 provides the build; TypeScript 5 with strict mode compiles it; Tailwind CSS 4 with the CSS-first @theme configuration styles it; and a shadcn/ui new-york component layer with 27 Radix primitives supplies interaction patterns. Animation is framer-motion 12, charting is recharts 2.15, fonts are self-hosted Geist families via next/font, and the native track is Capacitor 8.5.1 with splash-screen and status-bar plugins. Tooling is Bun-first: bun.lock governs installs (1,022 packages deep), Node 22 is pinned by .nvmrc and CI, and the deploy pipeline runs on GitHub Actions with the Vercel CLI prebuilt flow.

Layer	Package	Version	Audit note
Framework	next	16.1.3	Current major; static export path exercised daily by CI
Runtime UI	react / react-dom	19.2.3	React 19 stable line; no concurrent-feature misuse found
Language	typescript	^5.x	strict: true but noImplicitAny: false - see Ch. 8
Styling	tailwindcss	4.1.18	v4 CSS-first config in globals.css; tailwind.config.ts is dead v3 legacy
Components	@radix-ui/* (27)	mixed	18 of 48 generated shadcn files actually imported; 30 are dead weight
Motion	framer-motion	12.23.2	Used in 14 files; no prefers-reduced-motion strategy
Charts	recharts	2.15.4	Used in 6 files; duplicated into two async chunks (~300 KB waste)
Native	@capacitor/*	8.5.1	Full Android/iOS projects tracked; app id com.chacadom.keja
Dormant	prisma + @prisma/client	6.19.2	In dependencies; zero runtime importers; starter schema only
Dormant	next-auth	4.24.13	EOL v4 line, never imported; should be removed (audit trail noise)
Dormant	z-ai-web-dev-sdk	0.0.18	Declared, never imported; no AI key exists in the bundle

Table 8: Core dependency posture (locked versions at commit 86e2a66).

The deadwood problem

The stack assessment's most material finding is that roughly twenty declared dependencies are never imported anywhere in `src/`: `zustand`, `next-auth`, both TanStack packages (`react-query` and `react-table`), `zod`, the `dnd-kit` trio, `next-intl`, `date-fns`, `uuid`, `react-syntax-highlighter`, the MDX editor, and the `z-ai` SDK among them. These are scaffold carryovers from the platform that generated the original project. Each one costs install time in CI (the lockfile is 345 KB), inflates the audit surface for security scanners, and - more subtly - misleads any engineer or investor reading `package.json` into believing the platform uses `react-query`, `zod`, and `NextAuth` when it uses none of them. The pattern extends to code: 30 of 48 shadcn UI components (4,243 lines, including the two largest UI files in the repository) have zero importers, and `src/lib/db.ts` instantiates a Prisma client that nothing consumes. Chapter 26 prescribes the purge; it is the single cheapest high-impact action available to the team.

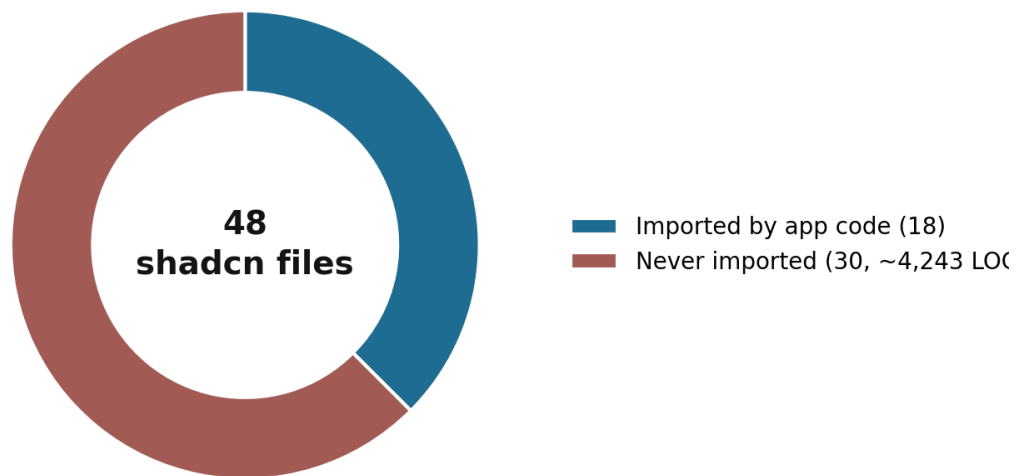


Figure 2: shadcn/ui primitives by import status: 30 of 48 generated files are never imported by application code.

Configuration posture

Build configuration mixes good discipline with quietly disabled guardrails. On the disciplined side: vercel.json pins the toolchain (Bun frozen installs), sets correct cache headers (service worker at max-age=0 must-revalidate; imagery on seven-day), and ships a full security header set including HSTS with a one-year max-age. On the disabled side: next.config.ts sets typescript.ignoreBuildErrors to true (compensated only because CI runs tsc separately), reactStrictMode is false, tsconfig weakens strict mode with noImplicitAny: false and allowJs: true, and the ESLint flat config explicitly turns off twenty-six rules - including correctness rules (no-undef, no-unreachable, no-fallthrough) that no team should ever surrender. The lint gate, as configured, passes by construction rather than by quality. The audit treats the configuration layer as a High-severity trust issue: the repository tells its tools to look away, and a future contributor has no signal telling them so.

```
// eslint.config.mis (excerpt) - rules disabled from the recommended sets
"@typescript-eslint/no-explicit-any": "off".
"@typescript-eslint/no-unused-vars": "off".
"react-hooks/exhaustive-deps": "off".
"no-console": "off". "no-undef": "off". "no-unreachable": "off".
// tsconfig.json
"strict": true. "noImplicitAny": false. "allowJs": true.
// next.config.ts
typescript: { ignoreBuildErrors: true }, reactStrictMode: false
```

Version currency and risk

Everything on the critical path is current: Next.js 16.1.3, React 19.2.3, Tailwind 4.1.18, Prisma 6.19.2 (dormant but modern), Capacitor 8.5.1 with targetSdk 36 on Android. The risk concentrates in the unused tail: next-auth 4 sits on the end-of-life v4 line with known 2024 CVE history (patched in this patch release, and unreachable in production since it is never imported - but it should not be in the tree at all), and the 0.x z-ai SDK is unaudited third-party code that the platform does not actually use. The deeper version risk is structural rather than per-package: with 81 declared dependencies and 1,022 installed packages, a static-export SPA is carrying a supply chain sized for a serverful application. Chapter 26's dependency purge reduces both the real and the perceived exposure.

7. Source Inventory and Code Metrics

What is actually in the repository

The tracked tree contains 435 files. The application source (src/) is 136 files and 33,253 lines: 92 components (48 shadcn primitives plus 44 product views across 18 feature folders), 32 library modules, 6 authored data modules, 3 app-router files, 2 hooks, and 1 config. Around it live 122 public assets (84 property photographs with jpg/webp pairs, 17 iOS splash screens, 6 icons, brand imagery, the service worker, the manifest, the offline page), complete Capacitor Android (77 files) and iOS (26 files) projects, 32 operational scripts (the Auto-Pilot pipeline, the sitemap generator, PWA asset generation, PDF document builders), 18 documentation files with rendered PDF editions, and the four GitHub Actions workflows. The repository pack weighs 23.4 MB - reasonable for the content, though Chapter 16 identifies roughly five megabytes of deployed imagery that no code references.

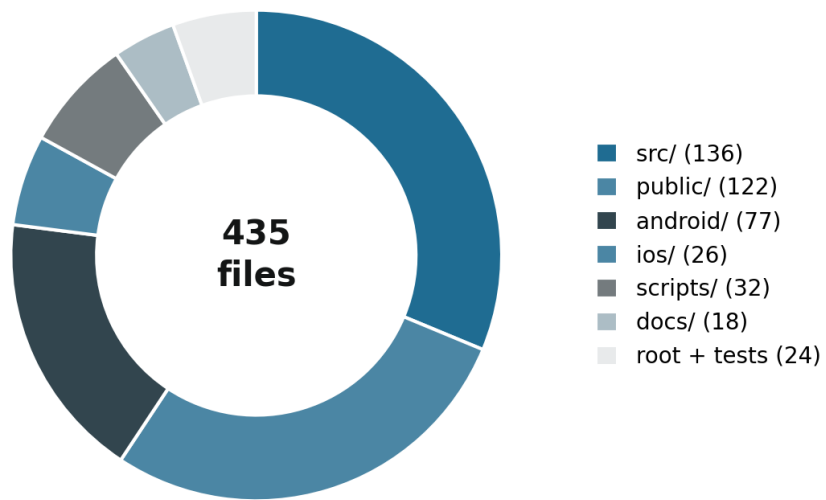


Figure 3: Distribution of the 435 tracked files across top-level directories.

Where the lines are

Line-count distribution tells the architecture's story at a glance. Components hold 45.9 percent of source lines and the authored data modules 26.6 percent - together nearly three-quarters of the codebase is presentation and content, which is exactly what a client-side marketplace should look like. The library layer (25.4 percent) carries the platform's real engineering: the router, the store primitive, the nine domain stores, the finance mathematics, the AI engine, verification, analytics, and CSV export. Within that layer, the largest files are also the most consequential - the conversational engine at 1,049 lines, the tokenization store context at 1,041, the authentication layer at 556 - and the audit's file-level findings in Part III concentrate there.

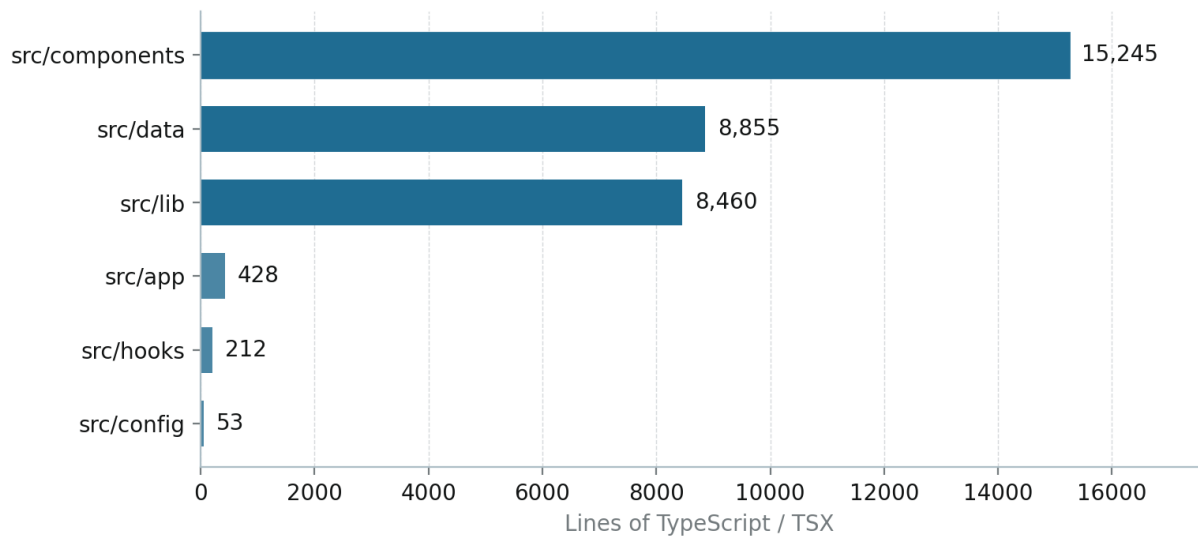


Figure 4: Lines of TypeScript/TSX by source area (33,253 total).

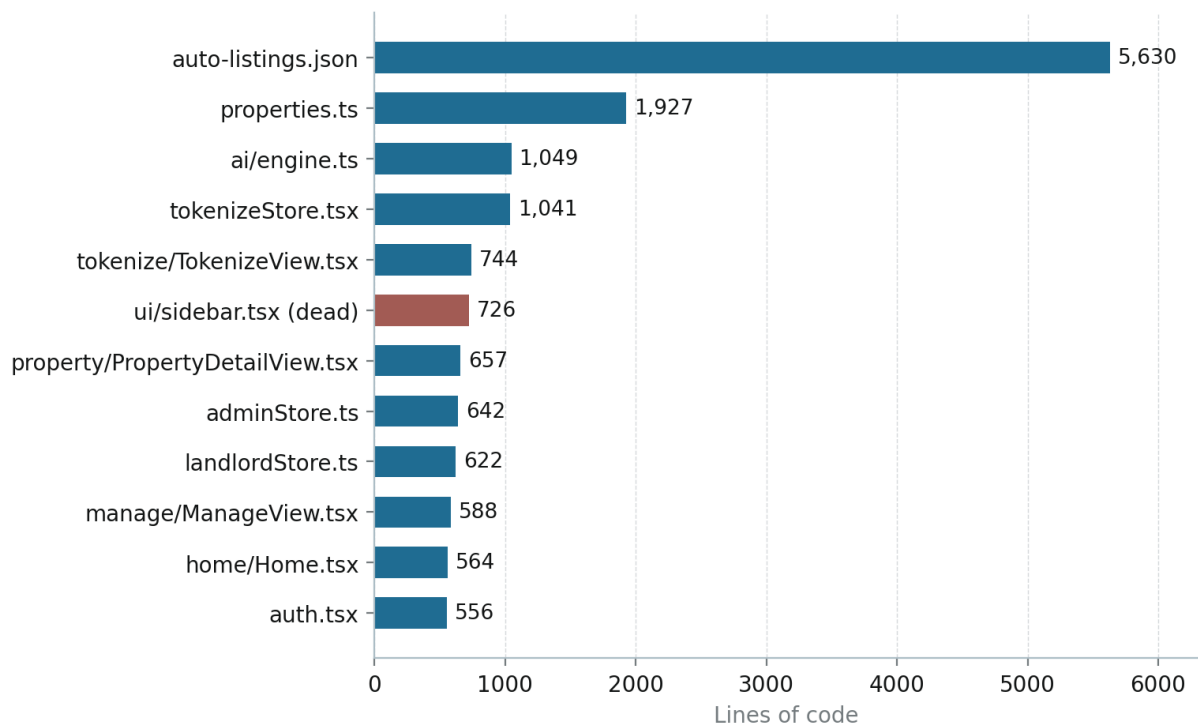


Figure 5: The twelve largest source files by line count; the dead sidebar component is flagged in red.

Size hotspots and what they mean

The hotspot list rewards a careful read. `src/data/auto-listings.json` at 5,630 lines is generated output committed to the repository - acceptable for a static pipeline, but it will grow linearly with inventory and Chapter 24 recommends a strategy before it reaches ten thousand lines. The seed catalog (1,927 lines) is authored content with real domain modelling in its types. The remaining hotspots split into two classes: files that are legitimately complex domains (the engine, the tokenization context, the property detail view, the landlord and admin stores) and files that are simply monolithic - `ManageView` renders a 523-line component function mixing metrics, alerts, dialogs, and tables in one closure. Chapter 26 sets a refactoring agenda for the second class, sized to the places where complexity actually hurts (re-testability and change confidence) rather than to a cosmetic line budget.

File	Lines	Disposition
<code>src/lib/ai/engine.ts</code>	1,049	Complex but cohesive; needs regression tests before any Phase 2 LLM work (Ch. 22)
<code>src/lib/tokenizeStore.tsx</code>	1,041	Monolithic context; KYC persistence is a Critical finding (Ch. 12)
<code>src/components/tokenize/TokenizeView.tsx</code>	744	Split presentation from order-book simulation
<code>src/components/ui/sidebar.tsx</code>	726	Dead code - delete (Ch. 26)
<code>src/components/property/PropertyDetailView.tsx</code>	657	Good structure; add lead-capture reality (Ch. 18)
<code>src/lib/adminStore.ts</code>	642	Sound; admin surface itself is ungated (Ch. 12)
<code>src/lib/landlordStore.ts</code>	622	Clean store pattern; ledger needs server truth in Phase 2
<code>src/components/manage/ManageView.tsx</code>	588	Refactor priority one: 523-line component function
<code>src/components/home/Home.tsx</code>	564	Static import inflates initial bundle (Ch. 16)
<code>src/lib/auth.tsx</code>	556	Contains the Critical DJB2 finding; Phase 2 auth seam

Table 9: The ten largest hand-written modules and their audit disposition.

Repository hygiene

Hygiene is mixed but improving. The .gitignore correctly excludes the sandbox's working directories, no .env file was ever committed (verified across all 106 commits), and the worklog discipline of the operating team is visible in the commit history. Against that: the tests directory contains three orphaned shell scripts that test a .zscripts platform directory which does not exist in the repository - dead weight that misrepresents the test story to any reader; scripts/prerender.mjs is a leftover from the Vite era referencing a dist/ directory that no longer exists; the dead tailwind.config.ts v3 file contradicts the actual v4 CSS-first configuration; and binary weight is carried in the form of six duplicate 939 KB iOS splash PNGs and committed PDF build artifacts under scripts/keja-docs. None of these is urgent; all of them are one afternoon of cleanup that materially raises the repository's credibility with outside engineers.

PART III - CODE-LEVEL ANALYSIS

8. TypeScript and Code Quality

The paradox of the type system

The most striking quality result in this audit is a paradox. On paper, the configuration has been weakened in three separate places: tsconfig sets strict true but disables noImplicitAny and enables allowJs; next.config.ts sets ignoreBuildErrors true; and the ESLint flat config turns off twenty-six rules including the TypeScript hygiene set. And yet the actual code contains **zero uses of the any type, zero as-any casts, zero @ts-ignore and zero @ts-expect-error** across all 33,253 lines - a discipline level the audit team rarely sees even in teams that enforce these rules with CI. The paradox resolves into a simple judgment: the code is clean, but the guardrails are off, which means the cleanliness is a property of the current authors rather than of the repository. The first contributor who leans on any, or the first refactor done under time pressure, will meet no resistance. Re-arming the configuration is therefore not remediation of a present defect - it is insurance on an asset that currently exists only by convention.

0

uses of any / as any

0

@ts-ignore / expect-error

13

non-null assertions

26

ESLint rules disabled

Escape hatches in use

The audit catalogued the remaining type escape hatches. Thirteen non-null assertions exist, clustered in the comparison feature (CompareBar, CompareView) and one in the AI chat view; each is guarded by a length check before the assertion, so they are low-risk today, but a small Property-or-undefined narrowing helper would eliminate the entire class. Two as-unknown-as double casts exist: one in the dead db.ts file, and one in src/lib/autoListings.ts line 86, where a 5,630-line JSON import is trusted blindly as typed state - the only genuinely risky cast in the codebase, since a malformed generated file would type-check happily and fail at runtime in front of users. Zod, which would solve exactly this, sits unused in the dependency list. The audit rates the JSON cast Medium and pairs it with a recommendation to validate generated data at the boundary in Chapter 26.

Patterns that carry the codebase

Several recurring patterns deserve explicit credit because they do disproportionate work. The **store primitive** (src/lib/store.ts) is a single generic useStore hook - localStorage read/write with a keja: prefix, a CustomEvent broadcast for cross-hook synchronisation, and seed fallbacks - reused by all nine domain stores; this is composition, not copy-paste. The **magic-number discipline** is real: the exchange peg (FX_KES_PER_USD = 129), the rental price floor, verification validity windows, trial wallet balances, and trust floors are all hoisted, named, and commented at their definition sites. The **header-comment culture** - module headers that explain why localStorage, why hash routing, why the trust caps exist - turns the codebase into its own documentation. And the finance module (src/lib/finance.ts, 270 lines) is genuinely well-built: correct amortisation and affordability mathematics with named Kenyan cost constants, and previously covered by the now-deleted test suite.

Smells and their severity

The smell census came back unusually clean on the classics - zero TODO/FIXME/HACK markers, two dev-only console statements, no commented-out code blocks - and concentrated instead on structural issues. Twelve functions exceed 150 lines, with ManageView's component function at 523 lines and the engine's respond routine near 850 including its data tables; these monoliths are the primary reason the codebase is hard to test incrementally. Thirty-plus call sites use Date.now() as a unique identifier where a single uuid helper (uuid is already installed, unused) would prevent collision-prone rapid actions. The useStore effect closes over its fallback argument with a [key] dependency list, so callers passing inline default objects get stale references - latent, harmless today, confusing later. And the cross-tab storage listener that keeps authentication coherent across tabs is implemented manually in auth.tsx rather than living in the shared primitive where every store could benefit.

Category	Count	Severity	Disposition
any / as any / ts-ignore	0	-	None to remediate; guard by re-arming config (Ch. 26)
Non-null assertions	13	Low	Add a narrowing helper in compare feature
as unknown as casts	2	Medium	One dead (db.ts); one live - validate auto-listings JSON
Functions over 150 lines	~12	Medium	Refactor ManageView, DealAnalyst, engine respond()
Date.now() identifiers	30+	Low	Central uuid helper (uuid already installed)
Dead UI components	30 files	High	Delete in the Ch. 26 purge
Dead modules / deps	1 + ~20	High	Delete db.ts; prune package.json
Inline format helpers	~10 files	Low	Consolidate on src/lib/format.ts
console statements	2	Low	Both dev-guarded by design

Table 10: Code smell census by category, with audit severity and disposition.

Git history as a quality signal

History quality is adequate but thin on process. Commit messages follow conventional-commit prefixes with 38 honest Auto-Pilot chore commits declaring their listing counts. Two legacy subjects break the convention; one earlier stray UUID-message commit was properly reset before it ever reached origin. The structural weaknesses are the absence of pull-request review (everything lands on main directly, so the pr-check workflow has never exercised its intended path) and the eight distinct author identities, including automated identities committing as the platform owner to satisfy Vercel Hobby plan attribution rules - a documented, understandable workaround that nonetheless weakens the audit trail separating human decisions from machine ones. The single-branch, no-review pattern is defensible at one maintainer; Chapter 26 recommends lightweight PR discipline beginning with the first multi-contributor change.

9. Component Architecture

The shape of the component tree

The application mounts through a disciplined shell: KejaApp composes the hash router, a global error boundary, a Suspense layer, the authentication and tokenization providers, and then the route outlet - surrounded by the navbar, demo banner, footer, floating WhatsApp entry, mobile tab bar, and install prompt. Below the shell sit 44 product views across 18 feature folders (property, trust, shell, ai, finance, invest, manage, common, admin, data, developer, diaspora, institutional, partners, tokenize, home, and the theme provider), with 31 of the 32 routes lazy-loaded for route-level code splitting. The only statically imported view is Home - a choice that puts Home, the framework, and framer-motion into the first-load critical path, examined further in Chapter 16.

Consistency across views is a real strength. Every view follows the same anatomy - a badge and title block, a max-width container, Lucide icons marked aria-hidden, Tailwind design tokens rather than ad-hoc values - which makes navigation between features feel like one product rather than a federation of experiments. Prop drilling is essentially absent: views pull state directly from the domain hooks (useAllProperties, useLandlordStore, useAuth, useTokenize, useSavedSearches), and the context layer is reserved for the four concerns that genuinely need it (router, auth, tokenization, internationalisation).

Monoliths

The component layer's principal weakness is a handful of views that grew into single-closure monoliths. ManageView renders one 523-line component function; DealAnalystView reaches 418; InvestorDashboardView 370; the properties browsing inner component 349. What these share is not just length but the mixing of four concerns in one closure: derived-metric computation, alert-rule evaluation, dialog state, and rendering. The cost is paid in testability (there is no seam to exercise the alert rules without mounting the world), in review friction, and in re-render scope - every state change in a 500-line closure re-runs the whole render function. The audit's refactoring prescription in Chapter 26 is deliberately narrow: extract the pure computation (metrics, alert evaluation, filters) into testable modules first, and leave visual restructuring alone until the extraction lands.

The shadcn layer

The generated primitive layer tells a story of ambition outpacing usage. Of 48 shadcn/ui files, application code imports 18 - dominated by button (37 files), badge (32), input (18), label (15), tabs (10), and progress (9). The remaining 30 files (4,243 lines) have zero importers: the entire sidebar system, the chart wrapper, cards, tables, dropdown menus, calendars, carousels, drawers, command palettes, and more. Meanwhile the team hand-rolls cards and menus with Tailwind classes in the views - competently, but outside the installed component system. The pattern is classic generated-scaffold inheritance: the scaffold assumed an admin-dashboard shape; the product that grew is a marketing-and-portal shape. The correct response is subtraction (delete the unused 30, and the four dependencies that exist only to serve them - embla-carousel, react-day-picker, react-resizable-panels, and the react-hook-form stack used only by the dead form.tsx), not guilt-driven adoption.

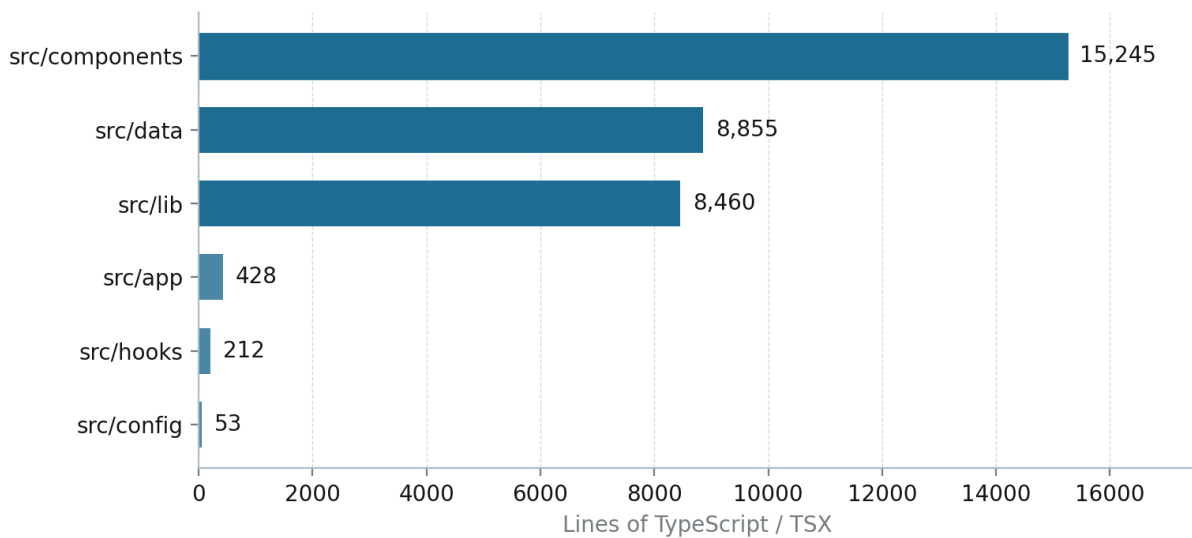


Figure 6: Component code dominates the source tree at 45.9 percent - appropriate for a client-side marketplace.

Cross-cutting components worth studying

Several components merit mention as exemplars for the team's own future work. `ErrorBoundary` wraps the route outlet with a user-facing retry UI and correct dev-only logging (its lack of production reporting is an observability gap, not a component defect). The `focus-trap` hook implements stacked-dialog-aware focus management with escape handling and focus restoration - genuinely sophisticated accessibility engineering. `InstallPrompt` implements the full `beforeinstallprompt` choreography with an iOS Share-menu walkthrough, dismissal cooldowns, and standalone-context suppression. `CompareBar` demonstrates the right way to lift shared interaction state into a hook with a floating tray. These are the patterns to replicate when building the Phase 2 surfaces; they are also the components that most deserve the first regression tests.

Assessment

The component architecture scores well on consistency, composition, and accessibility, adequately on performance-sensitive structure (lazy loading is in place; render-scope discipline is not), and poorly on inventory hygiene. Nothing in this layer blocks Phase 2 work. The priorities that emerge are: delete the dead 30 primitives and their four sole-user dependencies; extract pure logic from the four monolith views to make the upcoming test restoration meaningful; and hold the existing view anatomy conventions as law when new server-backed surfaces arrive, because the codebase's uniformity is the asset most easily lost and hardest to rebuild.

10. State Management and Data Layer

The state architecture

State management in Keja AI is a three-layer design that fits its constraints precisely. Layer one is build-time authored data: the seed catalog, Auto-Pilot JSON, and editorial modules, merged at runtime by the inventory layer into the property collection the views browse. Layer two is the generic persistence primitive: `useStore` in `src/lib/store.ts`, which gives any domain a namespaced `localStorage` slot, seed defaults, JSON serialisation, and a `CustomEvent` (`keja-store-change`) that keeps every hook consumer in sync within a tab. Layer three is the nine domain stores built on that primitive - landlord, tenant, admin, search, investor, developer, diaspora, professional, and valuation - plus two React contexts for authentication and the tokenization sandbox, and a small set of feature hooks (saved searches, notifications, alerts, compare) layered above the search store.

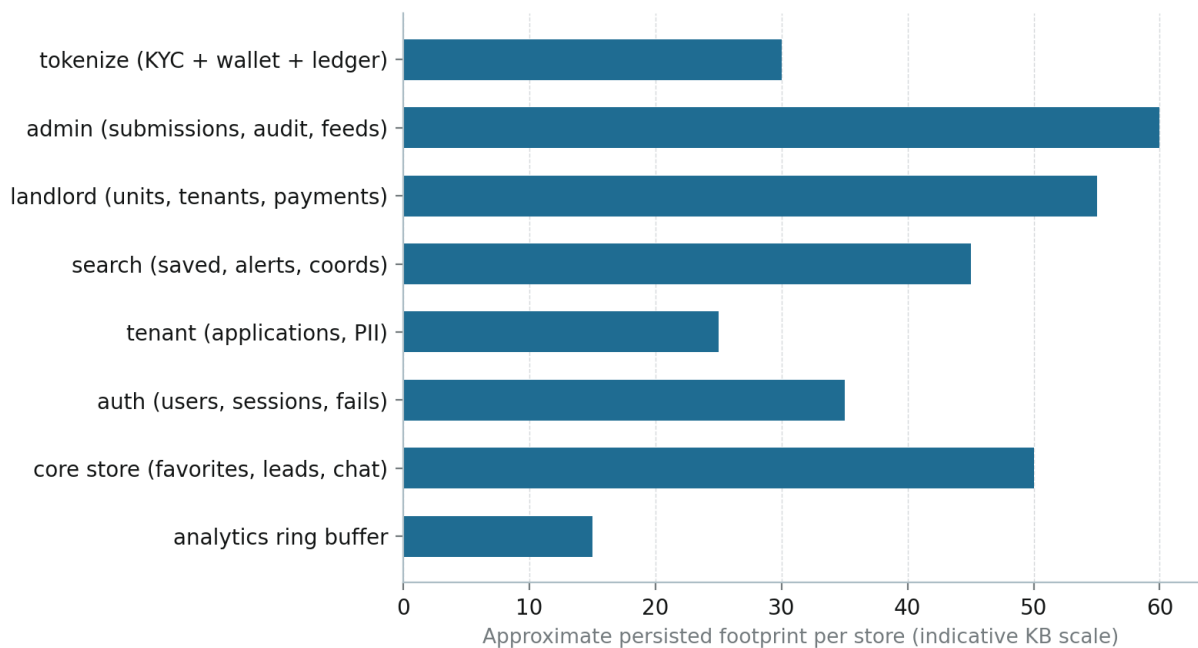


Figure 7: Indicative persisted footprint by domain store: the tokenization, admin, and landlord domains carry the heaviest client-side state.

Why this design works - and where it strains

The design works because it is honest about being single-device and single-tab-origin. The stores are documented as such in their headers; the demo surfaces declare their nature; bounded ring buffers (audit log at 500 entries, analytics at 200 events, listing signatures at 200) show real quota awareness. The design strains in three specific places the audit verified. First, the shared `read()` helper parses unvalidated JSON on every access - a corrupted or truncated write silently falls back to seed data, which surfaces as mysterious data loss and, in the admin or landlord domains, as quietly unreliable records. Second, cross-tab coherence exists only for authentication; a landlord with the dashboard open in two tabs will diverge state between them on every mutation. Third, the `useStore` effect dependency pattern (`[key]` only, fallback closed over) produces stale defaults for callers who pass inline seed objects - latent today, a trap for the next store author.

The data dictionary discipline

One practice in this layer deserves explicit praise because it quietly determines the cost of Phase 2: the platform maintains a data dictionary (`docs/DATA_DICTIONARY.md`, auto-generated from the source modules by a regeneration script) that documents every persisted shape - fields, types, semantics - across the datasets. Combined with the fact that every domain store's TypeScript interfaces already model what its server counterpart would need (a `RentalApplication` with income and references; a landlord ledger with payments and channels; a KYC record with document type and status), this means the Phase 2 migration is a mapping exercise, not a discovery exercise. The audit's Phase 2 schema recommendations in Chapter 24 were drafted directly against these interfaces; that is only possible because the client layer kept its contracts typed and documented.

The server-shaped gap

The gap between this layer and a real marketplace backend is exactly one seam wide, and it is worth enumerating what does not exist today: no fetch calls anywhere in `src/` (verified - zero network egress), no API client module, no query cache, no mutation queue, no optimistic-update reconciliation, no conflict model for the day two devices edit the same landlord ledger. The declared-but-unused `react-query` and `zod` packages are the scaffold's ghost of this future. The audit's position, developed fully in Chapter 24, is that the seam should be introduced at the store boundary: each domain hook gains an API-backed twin behind the same call signature, with `localStorage` retained as the offline cache that the service worker strategy already understands. That choice converts today's limitation (state lives in the browser) into tomorrow's feature (state works offline and syncs when online) - which is, not coincidentally, the right product promise for the Kenyan connectivity context the platform was designed for.

Domain store (client)	Owns today (localStorage)	Phase 2 counterpart (server)
auth context	Users, sessions, lockout counters, demo accounts	Auth service: HttpOnly sessions, argon2, OIDC
landlordStore	Units, tenants, rent ledger, maintenance tickets	Listings, tenancies, payments (M-Pesa reconciled), tickets
tenantStore	Applications with income, employer, references	Application API + landlord notification pipeline
adminStore	Submissions, audit log, feeds, settings, reports	Moderation queue, audit service, feed registry
searchStore	Saved searches, alerts, area coordinates, compare	Per-user saved searches, server-side alert matching
investorStore	Portfolio positions, watchlists	Portfolio service with market data feed
tokenize context	KYC records, trial wallet, ledger, orders	CMA-sandboxed custody and settlement service
diaspora / dev / pro	Diaspora briefs, developer projects, pro workspace	Portal APIs gated by role
analytics (local)	200-event ring buffer, taxonomy-typed	Privacy-preserving product analytics (Plausible-class)

Table 11: The nine domain stores and their Phase 2 server counterparts.

Assessment

This is the layer the audit scores highest on design-for-purpose and highest on migration readiness. The store primitive is clean, the domain decomposition is sensible, the contracts are typed and documented, and the quota discipline shows forethought. The Medium findings (unvalidated reads, single-tab events, stale-fallback hazard) are real but bounded. The strategic point is that nothing here needs to be thrown away: the layer that was built to work without a server is, properly extended, also the layer that makes the arrival of a server safe - provided the introduction is done one domain at a time with the offline story preserved deliberately rather than accidentally.

11. AI Capability Assessment

What the AI actually is

The platform's most important naming decision - and its most dangerous expectation gap - is the phrase **Ask Keja**. The AI advisor is not a language model. It is a 1,049-line deterministic conversational engine (`src/lib/ai/engine.ts`) that classifies user intent against a taxonomy, resolves entities (approximately fifty Nairobi-area aliases and colloquialisms), performs investment mathematics over the live inventory, and composes responses from parameterised templates - in English, Kiswahili, and French - with a simulated 420-650 millisecond latency so the experience feels conversational rather than instantaneous. A companion module (`marketIntel.ts`) answers natural-language questions over the merged property collection. No LLM is called, no SDK is imported, no key exists in the bundle. This is, the audit stresses, a defensible and even clever architecture for Phase 1: it is free, offline-capable, latency-free, hallucination-free by construction, and its answers are auditable line by line - properties a Kenya-market trust product should value over conversational glamour.

The engine's architecture

Internally the engine is a pipeline: normalisation and language detection; intent classification across a rule set covering search, affordability, investment math, area questions, process questions (deposits, leases, Ardhisa), and platform questions; entity extraction against the alias tables; response composition with the labelling system (FACT for verified data, ESTIMATE for computed figures with stated assumptions, ASSUMPTION for user-supplied premises, REPORTED for third-party claims); and a closing layer that can attach live property cards inline. The most safety-relevant component is the **escalation guard**: legal, tax, and valuation questions are detected and routed to human handoff with WhatsApp escalation rather than answered - the exact behaviour a rental platform needs to avoid giving legal advice. This guard previously had a dedicated test suite (`escalation.test.ts`, 58 lines) that was deleted in the rebuild and is first in line for restoration in Chapter 22, because it is the single behaviour in the AI surface whose silent regression would carry real user harm.

Capability	Implementation	Assessment
Intent classification	Rule-based taxonomy, trilingual	Deterministic; needs regression table tests
Area entity resolution	~50 Nairobi aliases incl. colloquial forms	Strong localisation moat; brittle to typos by design
Investment mathematics	Shared with finance module (single source)	Correct per Ch. 8 review; untested (Ch. 22)
Answer labelling	FACT / ESTIMATE / ASSUMPTION / REPORTED	Rare, trust-building; keep under any LLM upgrade
Escalation guard	Legal/tax/valuation to human handoff	Safety-critical; restore tests first
Property card attachment	Inline results from merged inventory	Good grounding behaviour
Language support	EN / SW / FR dictionaries	UI-level depth varies; content mostly EN
Memory	Chat history in localStorage per user	Single-device by architecture

Table 12: Ask Keja capability inventory as implemented.

The expectation gap and how to close it

The risk is not the engine; it is the label. A product named and marketed as AI that answers with template determinism will read as search-with-a-chatbox to the first sophisticated user, while a product that quietly swaps in an LLM inherits every hallucination and liability risk the current architecture elegantly avoids. The audit's recommendation is a two-sided honesty play: in the near term, let the UI say what the engine is (a rules-plus-data assistant with cited labels - the claims register already provides the vocabulary); in Phase 2, when and only when the escalation guard has regression tests and the server seam exists, integrate an LLM **behind the existing engine interface** - the engine's contract (intent in, labelled response out) is precisely the adapter shape an LLM needs, with the rule engine retained as the deterministic fallback for offline use and for the question classes where the answer must never vary. That sequencing preserves every property that makes the current design trustworthy while opening the capability path the brand implies.

Adjacent AI surfaces

Two further surfaces carry the AI branding and deserve the same honesty treatment. The **Deal Analyst** performs on-device document pre-screening - heuristic checks on uploaded documents against expected patterns, with results framed as screening rather than verdicts; it is genuine logic, modestly scoped, and correctly labelled. The **trust scoring** that colours every listing card is deterministic arithmetic over verification evidence and listing attributes with hard caps (synthetic listings capped at 94, verified floors enforced) - this is arguably the platform's most valuable "AI" asset precisely because it is explainable to a sceptical landlord, and the audit's Phase 2 schema keeps its inputs first-class so the explanation survives the database migration.

PART IV - SECURITY, DATA AND COMPLIANCE

12. Security Audit

Posture in one paragraph

Keja AI's security posture is defined by a single structural fact: there is no server. The static export eliminates the entire server-side OWASP surface - no injection, no broken file access, no SSRF, no deserialisation, no misconfigured endpoints - and the audit verified the pleasant corollaries: zero secrets in the client bundle, zero network egress from application code, no third-party trackers, no cookies, and a service worker that caches only same-origin GET responses with 200 status. The platform also gets the header basics right that most forget: X-Frame-Options DENY, nosniff, a strict referrer policy, a permissions policy, and a one-year HSTS. What remains is the client-side trust problem: every control that matters - authentication, roles, throttling, data ownership - executes in a context the user fully controls, and two of those controls are implemented in ways that would be unacceptable if a real user relied on them.

Critical finding C-1: credential storage

The authentication layer (`src/lib/auth.tsx`) accepts email-and-password registration from any visitor and stores those credentials in `localStorage` after hashing with **DJB2** - a non-cryptographic, unsalted, reversible-by-brute-force string hash from 1997 - in the format `k1$hash$length` under the `keja:pw` key. The demo accounts (`admin/agent/investor` at `keja.ai` with trivial passwords) are hardcoded in source at lines 197-202 and 395-399, and the simulated Google accounts are equally public. For the demo phase this is labelled and survivable. The criticality arises from combination: registration is not demo-gated, the platform is publicly marketed, and password reuse across services is the norm for most users - so a real user's real reused password can end up trivially recoverable from their own browser storage (and from any device or extension that can read it) by anyone with ten lines of script. The audit's remediation is staged: immediately block or loudly warn on real-looking password registration in demo builds; before any real authentication, migrate to server-side sessions with `argon2` hashing and `HttpOnly` cookies (Chapter 24 specifies the seam).

Critical finding C-2: KYC data collection

The tokenization flow collects national identification - idType, idNumber, and a source-of-funds declaration - into plaintext localStorage (tokenizeStore, lines 188-196 and 246-252; the form surfaces at TokenizeView lines 310-339). There is no encryption at rest, no retention limit, no data-subject notice at the point of collection, and no erasure path. Under the Kenya Data Protection Act 2019, national ID numbers and financial-source data are sensitive personal data; collecting them without registration as a data controller with the ODPC, and without the Act's safeguards, is a compliance exposure that activates the moment a real user types real values - which the current UI happily permits. The platform's own documentation half-acknowledges this by deferring real KYC to the CMA sandbox process, but the form is live and ungated. Remediation: do not persist identifier values in the demo (collect-and-discard, or gate the flow), and treat the collection point itself - notice, purpose, retention - as a first-class Phase 2 requirement rather than a compliance afterthought.

High finding: the ungated admin console

The #/admin route renders the platform administration console - listing submissions, the audit log, partner applications, feed configuration, platform settings - to any visitor who types the hash. The authentication context exists, the role model exists, and the account view consumes them; the admin view consumes neither. In a localStorage world the console cannot actually mutate shared state (it edits only the local browser), so the damage today is cosmetic: a demo visitor can see the moderation surface and be misled about what is real. But the pattern is the kind that survives migration - a view that never learned to check the door - and the audit flags it High precisely because Phase 2 will make that door load-bearing. The fix is five lines today (render a sign-in gate when isAdmin is false) and a server-side authorisation check the day the console gains a real API behind it.

High finding: no Content-Security-Policy

The one header that meaningfully defends a data-driven SPA - Content-Security-Policy - is absent. The application loads zero external origins (fonts are self-hosted, no CDNs, no trackers), which makes Keja AI the easiest possible CSP deployment: a default-src 'self' with script-src 'self' would ship in one vercel.json line and would have blocked, for example, any future supply-chain script that ever slipped into a dependency. The audit recommends deploying it in report-only mode first (to catch the odd inline style), then enforcing - and pairing it with HSTS preload submission once the domain has served the header continuously. While in the file, the Permissions-Policy can be tightened: geolocation is currently allowed for self but the application never uses it.

What the audit verified as clean

- **Secrets sweep:** no ghp_/vcp_/nfp_/sk-/AKIA/AIza patterns in any tracked file, and git history shows exactly one .env.example (public VITE vars) ever committed, later deleted in the rebuild.

- **XSS surface:** exactly one dangerouslySetInnerHTML (the shadcn chart wrapper's CSS-variable theming, no user data); ReactMarkdown renders without rehype-raw so chat and agent text is escaped; no eval or Function constructors; external links carry noopener noreferrer.
- **CSV export hardening:** src/lib/csv.ts neutralises formula injection (=, +, -, @, tab, CR prefixes) on user-controlled exports - exemplary and rare.
- **Service worker safety:** same-origin GET-only, 200-only caching, versioned caches self-evicting on activate - no cache-poisoning pattern.
- **CI script injection:** workflows pass untrusted input via environment rather than string interpolation, per GitHub's own guidance.
- **Rate limiting:** client-side five-failures-per-minute lockout exists; bypassable by clearing storage, but present and commented.

OWASP mapping

ID	Severity	Finding	OWASP category
C-1	Critical	DJB2 password hashing into localStorage; open registration accepts real credentials	A02 Cryptographic Failures
C-2	Critical	KYC national ID and source-of-funds persisted plaintext client-side	A04 Insecure Design
H-1	High	#/admin console renders with no authentication or role check	A01 Broken Access Control
H-2	High	No Content-Security-Policy header	A05 Security Misconfiguration
H-3	High	Demo credentials and simulated Google accounts hardcoded in source	A07 Identification Failures
M-1	Medium	Unused next-auth v4 (EOL line) and z-ai 0.x SDK in dependency tree	A06 Vulnerable Components
M-2	Medium	Client-side throttling and sessions bypassable via storage clear	A04 Insecure Design
M-3	Medium	Unvalidated JSON.parse of persisted state on every read	A08 Software and Data Integrity
M-4	Medium	Hardcoded seed PII in real-contact format across auth and stores	Privacy / hygiene
M-5	Medium	Caddyfile dev proxy exposes arbitrary-port forwarding pattern	A05 (dev-only)
L-1	Low	Permissions-Policy allows geolocation that the app never uses	A05
L-2	Low	ignoreBuildErrors and reactStrictMode disabled in next.config	A05

Table 13: Security findings register mapped to OWASP Top 10 (2021) categories.

Threat model

To make the register actionable rather than merely declarative, the audit modelled the platform against a lightweight STRIDE pass over its two trust boundaries: the browser session (the only boundary that exists today) and the future API seam (Chapter 24). For each threat class the table records the current exposure and the architectural control that closes it - which doubles as a preview of why the Phase 2 seam is shaped the way it is. The reading is blunt but clarifying: today, every class that depends on server-verifiable truth is open not because of a bug but because the boundary is absent, which is why the remediation is architectural and why the Chapter 27 P0 items gate the marketing push rather than the other way round.

Class	Threat in scope	Today (browser boundary)	Control at the API seam
Spoofing	Attacker impersonates a user or admin	Trivial: roles are client state (F-05, F-01)	Server sessions + role checks on every endpoint
Tampering	User edits own ledger, trust data, application records	Trivial: all state is user-mutable	Server-side records; client cache is advisory
Repudiation	Actor denies a moderation or payment action	Audit trail is local and editable	Write-once AuditEntry table (Ch. 25)
Information disclosure	Credentials and KYC readable from storage	F-01, F-02 (Critical)	Server-side hashing; encrypted identifiers
Denial of service	Abuse of future endpoints or the AI seam	N/A today (no endpoints)	Per-identity and per-IP limits at the boundary
Elevation of privilege	Visitor mints admin rights	Possible via storage edit	Roles issued and verified server-side only

Table 14: STRIDE threat model across current and Phase 2 boundaries.

Assessment

For a platform with no server, Keja AI is unusually defensible: the attack surface is one static bundle, it leaks nothing, and its hygiene habits (noopener discipline, CSV hardening, SW safety) reflect genuine care. The two Critical findings share a shape - mechanisms that were written for demo users being exposed to real users - and both are cheap to defuse now and expensive to defuse after trust is established. The audit's security verdict: **secure as a demonstration, one release away from safe as a product**, with the gap concentrated in exactly the places Phase 2 already needs to build (authentication, KYC, admin) - which is the best possible timing for the fixes to land.

13. Data Model and Persistence

Two data models, one of them imaginary

The repository contains two data models that have never met. The real one lives in the client: roughly forty TypeScript interfaces across the store and data modules, documented field-by-field in the data dictionary, that describe properties, verification evidence, trust signals, tenancy applications, rent payments, maintenance tickets, KYC records, wallet ledgers, orders, audit entries, and feed connections. The imaginary one lives in `prisma/schema.prisma`: the untouched Prisma starter schema with its `User` and `Post` models - `authorId` an orphan string with no relation, no enums, no indexes beyond the implicit ones - served by a SQLite datasource and a `PrismaClient` singleton (`src/lib/db.ts`) that nothing imports. The `package.json` even ships a `db:push --accept-data-loss` script against this scaffold. The audit's finding is not that dormant scaffolding exists; it is that the distance between the two models is the true measure of Phase 2's data work, and that distance is currently unmapped in the one artifact (the schema) where it matters most.

Domain concept	Client-side model (real)	Prisma schema (scaffold)
Property / listing	Property with verification, trust, yields, pricing (27 seeds + 60 auto)	Absent
Tenancy	RentalApplication with income, employer, references	Absent
Payments	RentPayment with channel (mpesa/bank/cash), receipts	Absent
Users and roles	User with role user/agent/admin, audit trail	User: email + name only, no password, no role
KYC	KYCRecord with idType/idNumber/source OfFunds/status	Absent
Tokenization	Asset, wallet, ledger entries, orders	Absent
Moderation	Submission, report, audit entry, feed connection	Absent (Post model irrelevant)
Relations	Composed in interfaces and merge logic	<code>authorId</code> string, no <code>@relation</code> , no cascade
Constraints	Runtime discipline (trust caps, uniqueness by id)	None beyond unique email

Table 15: Domain coverage: client interfaces versus the Prisma scaffold.

The persistence reality

What the platform actually persists today is a well-organised set of twenty-eight `localStorage` keys (enumerated in Appendix B) - session and credential state for authentication; favourites, leads, chat history, saved searches, compare lists, and notifications for the core store; per-domain records for landlord, tenant, admin, developer, diaspora, professional, and investor state; the tokenization blob (`keja-tokenize-v1`) carrying KYC and wallet; a 200-event analytics ring; and assorted UI flags. The discipline around them is better than typical: a single namespaced accessor, bounded buffers where growth is unbounded in principle, and seed fallbacks that make a cleared browser self-heal to the demo experience. The limits are equally structural: five megabytes total quota, no cross-device truth, no backup, no concurrent-edit semantics, and user-mutability that makes every value advisory rather than authoritative.

Migration gap analysis

The audit assessed the gap between current persistence and the Phase 2 requirement along five axes. **Schema:** absent - the real domain must be authored in Prisma from the data dictionary before any backend code exists, or the API will crystallise around ad-hoc shapes. **API contract:** absent - no typed endpoint definitions, no error taxonomy, no pagination or filtering conventions. **Sync strategy:** absent - no offline queue, no conflict model, no idempotency for the day a landlord records rent offline and re-syncs. **PII handling:** absent - no encryption-at-rest design for identifier and income fields, though the Phase 2 database layer makes it straightforward (column-level encryption or vaulted identifiers). **Backfill:** trivial - by design, no real user data exists to migrate, which is the one unambiguous benefit of the `localStorage` phase: the schema can be authored correctly once, from documentation, without a single migration script from a legacy store.

Recommended schema shape

Chapter 24 presents the full target architecture; the data-model core of it deserves preview here. The recommended Prisma schema is authored from the data dictionary with the following properties: PostgreSQL as the datasource (the SQLite scaffold is dev-grade and the ecosystem advantages of Postgres - row-level security options, JSONB for flexible verification evidence, mature managed hosting - matter at exactly this scale); enums for every TS union (`PropertyType`, `Purpose`, `Availability`, `ClaimStatus`, `KYCStatus`, `PaymentChannel`, `SubmissionState`) so the database enforces what TypeScript currently asserts; unique composites where the client currently relies on discipline (unit plus period on rent payments, property plus investor on holdings); explicit cascade rules for the ownership graph (user to applications, listing to evidence); soft-delete and audit columns on the moderation-relevant tables; and the client's own audit-log event taxonomy promoted to a first-class `AuditEntry` model so the Phase 2 platform keeps the transparency property the demo already advertises.

Assessment

The data layer scores as a paradox worth stating precisely: **persistence engineering discipline is high; persistence itself is the platform's single missing organ.** The client-side modelling is genuinely good - typed, documented, quota-aware, seeded - and it constitutes an accidental but complete specification for the server schema. The dormant Prisma scaffold is harmless except as a signal: it implies a database exists. The audit recommends either authoring the real schema as the first Phase 2 commit (preferred - it is the contract everything else hangs from) or deleting the scaffold until that day (acceptable - it removes a false implication). What it should not do is ship Phase 2 incrementally with the starter models still in place, because the first ad-hoc endpoint will harden around them.

14. Privacy and Regulatory Compliance

The privacy posture is a strength - until data gets real

By the standards of consumer web platforms, Keja AI's privacy posture is close to exemplary: no third-party trackers, no cookies (the single sidebar-state cookie in the repository lives in dead code), no network egress at all, analytics that stay on-device in a bounded ring buffer with a published taxonomy, and no account required to browse. A user of the demo platform is, from a data-flow perspective, entirely private. This is partly design virtue and partly architectural consequence - a static bundle has nowhere to send data - but the effect is the same, and it aligns neatly with the trust-first branding. The compliance question begins where the demo ends: at the points where the platform collects identity-grade and financial-grade data into that private-by-accident context.

Kenya Data Protection Act 2019 exposure

The Kenya DPA 2019 requires data controllers and processors processing personal data of persons resident in Kenya to register with the Office of the Data Protection Commissioner (ODPC), and imposes obligations around lawful basis, purpose specification, minimisation, security, retention, and data-subject rights. Three platform surfaces engage the Act today. The KYC form collects national ID or passport numbers and source-of-funds declarations - sensitive personal data in anyone's reading - into plaintext browser storage with no notice, purpose statement, or retention limit. The tenant application collects monthly income, employer, and referee contact details. And the registration flow accepts real email-plus-password credentials. None of these is unlawful to display; all of them become regulated processing the moment they hold a real person's real data, and the current implementation would satisfy none of the Act's security or notice expectations in that moment. The audit's finding is timing-sensitive: this is cheap to fix before real users and reputationally expensive after.

Surface	Data collected today	Remediation stance
Tokenization KYC form	idType, idNumber, sourceOfFunds, plaintext localStorage	Do not persist identifiers in demo; add notice, purpose, retention before any real collection; encrypt at rest server-side in Phase 2
Tenant application	Monthly income, employer, references, phone	Notice at collection; minimise to what landlords act on; server-side storage with role-gated access in Phase 2
Account registration	Email, password (DJB2-hashed locally)	Gate or warn on real credentials in demo; real auth via server sessions (Ch. 24)
On-device analytics	Local events only, 200-event ring, no egress	No DPA exposure; preserve the local-first property when remote analytics arrive
Seed data in source	Real-format names, phones, emails across auth and stores	Replace with reserved-domain fictional data (@example.com, 07XX ranges)
WhatsApp handoff	Prefilled listing reference via wa.me link	No personal data in URL beyond listing id - acceptable; keep refs non-identifying

Table 16: DPA engagement points and the audit's remediation stance.

Registration and governance

When Phase 2 activates real data processing, the platform will need ODPC registration as a data controller (the process is an online registration with category declarations), a short privacy notice at each collection point - the current UI has none - and an internal data map that, conveniently, the data dictionary already half-provides. The tokenization ambitions add a second regulator: the platform's own documents track the CMA regulatory sandbox (three sandbox-related PDFs are committed under docs/cma, covering a testing plan and safeguards for a Karen waterfront tokenization quotation). The audit is not qualified to predict CMA outcomes, but notes the sequencing dependency plainly: collecting KYC data ahead of sandbox admission and ODPC registration is the single most avoidable regulatory risk the platform currently carries, and the fix is a demo-mode gate rather than a compliance programme.

Assessment

Privacy engineering earns the platform real credit: the local-first architecture, absent trackers, and honest labelling would satisfy most of the Act's technical-security spirit by default if the data were server-side. The compliance gap is concentrated, small, and fully enumerated: three collection surfaces lacking notice and safeguards, plus seed data hygiene. The audit schedules the demo-mode gating and fictional-data replacement in the first 30 days of the action plan (Chapter 27), and folds the notice, registration, and encryption work into the Phase 2 architecture milestones where they belong - as first-class requirements, not annexes.

PART V - DELIVERY AND USER-FACING DISCIPLINES

15. Progressive Web App Assessment

Installability verdict

Keja AI is genuinely installable on both major mobile platforms, and the implementation quality places it in the top tier of PWAs the audit team has reviewed. The manifest is complete and modern: a stable id, standalone display with a display_override chain, a navigate-existing launch handler, portrait orientation, branded theme and background colours, app categories, three app shortcuts (Properties, Ask Keja, Deal Analyst), and - the detail most teams skip - screenshot entries with form_factor declarations for both narrow (1080x2340) and wide (1920x1080) presentations, which unlocks Chrome's rich install dialog. Icons cover the required matrix: 192 and 512 PNGs for any, maskable variants at both sizes with drawn safe zones, an 180-point Apple touch icon, and both a 64-point PNG and an SVG favicon for browsers.

Criterion	Android	iOS	Evidence / note
Manifest served, parseable, with icons	Pass	n/a	manifest.webmanifest 2.1 KB, 6 icon entries
HTTPS with service worker + fetch handler	Pass	Pass	Vercel TLS; sw.js 109 lines, three-tier strategy
beforeinstallprompt capture and prompt UX	Pass	n/a	InstallPrompt 188 lines, 14-day dismissal cooldown
iOS Add to Home Screen walkthrough	n/a	Pass	Three-step illustrated guide in Share menu
Standalone chrome and status bar treatment	Pass	Pass	apple-mobile-web-app-capable + black-translucent; legacy meta belt-and-braces
Splash / launch screens	Pass	Pass	17 device-class startup images, 750x1334 to 2048x2732
Offline fallback	Pass	Pass	offline.html + inline Response final fallback
Maskable icon safe zone	Pass	n/a	generate-pwa-assets.mjs draws safe zone deterministically
Update flow	Warn	Warn	skipWaiting is silent; no new-version toast
Orientation flexibility	Warn	n/a	portrait-primary lock blocks landscape/tablets

Table 17: PWA installability checklist against current platform criteria.

The service worker

The service worker is the platform's best single file. It implements a three-tier strategy: immutable `/assets/` requests are cache-first with 200-only caching; images are cache-first in a versioned cache; and navigations race the network against an eight-second timeout, falling back through the cached page, the cached index shell, the offline page, and finally an inline Response - so the app never hard-fails offline. Cache writes are wrapped in `event.waitUntil` with a documented rationale (bare promise chains can be killed mid-write), the activate handler evicts every cache not matching the current version string, and `skipWaiting` plus `clients.claim` make rollouts immediate. The versioning story is the standout detail: a build script SHA-256-hashes the entire out/ tree and stamps the digest into the worker source, giving content-addressed cache busting that cannot drift - and `vercel.json` serves the worker itself with `max-age=0 must-revalidate` so updates are never stranded at the edge.

Gaps in an otherwise excellent implementation

Three gaps remain, all Medium or below. The update flow is silent: when a new version arrives, the worker takes over immediately with no user-facing signal, which is disorienting in a long-lived installed session - a toast with a reload affordance is the standard remedy and costs an afternoon. The orientation lock (portrait-primary) prevents landscape and large-tablet use of a property-browsing product whose comparison and dashboard views benefit from width; the audit recommends removing the lock and testing the shell at landscape aspect ratios. And `offline.html` still carries the retired gold-on-cream palette rather than the green brand system declared in the manifest - cosmetic, but it is the one screen where brand continuity matters most, because it is shown precisely when the platform feels broken.

iOS specifics

The iOS treatment deserves its own paragraph because it is where most PWAs quietly fail. The layout ships the full metadata set (capable, black-translucent status bar, viewport-fit cover with safe-area insets honoured in the shell's pt-safe/pb-safe utilities), and the seventeen startup images cover every current device class from 750x1334 to 2048x2732 - 2.2 MB of PNGs that most teams never generate. Safari ignores the fetch-handler caching tier for its own reasons, but the app shell still lands in the origin cache, and the offline page renders from the inline fallback. The install walkthrough in the prompt component teaches the Share-menu dance with illustrations, and the prompt correctly suppresses itself in standalone and Capacitor contexts. Net: an iOS user can add Keja to a home screen and receive a credible app-like experience - which is exactly the distribution story for a market where app-store discovery is weak and WhatsApp sharing is strong.

Assessment

PWA execution is a platform strength to protect, not a gap to close: the audit score is 4.5 of 5, with the half point held back only by the update-flow and orientation items. The strategic note that belongs here rather than in Chapter 16: installability is the platform's compensation strategy for its SEO darkness (Chapter 17) - the PWA-first, share-first distribution bet only pays if the installed experience is impeccable, because there is no search funnel backing it up. Every incremental investment in the installed experience (update toasts, offline listing caching of the property JSON, background sync when Phase 2 arrives) directly strengthens the one acquisition channel the current architecture actually supports.

16. Performance Assessment

Measured bundle composition

The audit measured the production artifact directly (out/, built with Turbopack and deployed). The first-load critical path - the nine script chunks referenced by index.html - totals 611 KB raw, about 183 KB gzipped, plus 145 KB of CSS (22 KB gzipped) and the self-hosted Geist fonts (164 KB across eleven woff2 files, two preloaded). With the HTML shell that is roughly 810 KB raw / 240 KB compressed before imagery - acceptable for the 3G-to-4G reality of the target market, and materially better than the platform's incumbent competitors ship, but with two avoidable inefficiencies embedded in it.

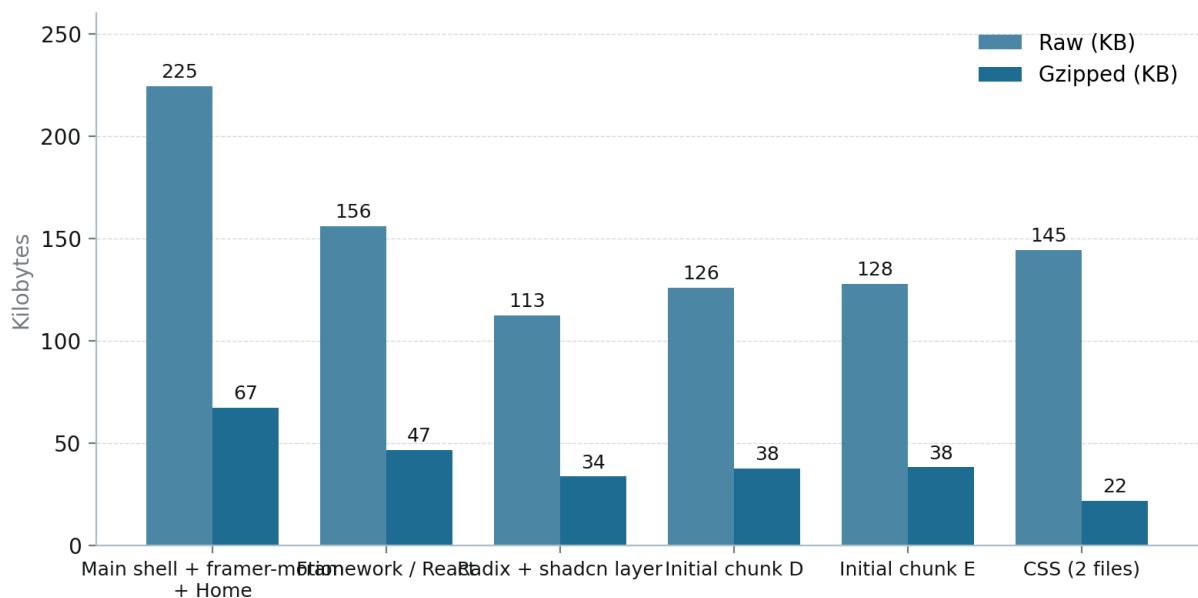


Figure 8: First-load chunk composition from the production build: raw versus gzipped sizes.

The recharts duplication

The single largest avoidable cost is that the recharts library ships twice. Two lazy chunks (2b20b3ee and 4de4ce44) each contain the full recharts/d3 dependency tree - 388 KB apiece, roughly 775 KB of raw JavaScript between them - because the bundler did not extract the shared module for the two asynchronous routes that import it (the market data view and the investor dashboard / deal analyst pair). A manual shared chunk, or routing all chart usage through one lazily-loaded module, removes roughly 300 KB from the lazy surface at zero product cost. The audit ranks this Medium severity - it does not touch first load, but it doubles the download for exactly the power users (investors) whose retention the platform most wants.

Imagery: the five-megabyte ghost

The audit cross-referenced every image reference in source against the deployed asset tree and found that all 132 property-image references resolve to .jpg files - not one .webp variant is referenced anywhere in src/, despite every property shipping both formats side by side. The WebP set is approximately five megabytes of dead weight on the CDN, and - the more actionable inverse - users are being served the heavier format on every listing card. Additionally, roughly 876 KB of brand imagery referenced only by PDF documents ships in the web bundle. No srcset, picture element, or DPR handling exists anywhere: full-resolution JPGs serve to every device class. The remediation is mechanical - swap references to the WebP variants (a 30-60 percent size reduction on identical quality), delete the unreferenced brand set from public/, and add width/height attributes to eliminate layout shift on image load.



First-load waterfall

Reconstructing the first visit from the artifact: the HTML shell (22.1 KB) references the nine critical chunks, two stylesheets, and two preloaded font files; the shell renders the branded loading splash until React mounts and the hash router resolves the initial view. The waterfall below is the honest budget the audit recommends optimising against - and the reference point for the P1 image work, since imagery lands after paint on listing-bearing routes.

Resource	Raw	Gzipped	Notes
HTML shell	22.1 KB	~6 KB	Includes RSC flight data for the splash
Critical JS (9 chunks)	611 KB	183 KB	Includes Home + framer-motion (F-item: lazy Home)
CSS (2 files)	144.6 KB	22 KB	Tailwind 4 output with token layer
Fonts (11 woff2; 2 preloaded)	164 KB	~150 KB	Geist + Mono, self-hosted, no third party
First listing hero image	~350 KB	n/a (JPEG)	Full-size JPG; WebP sibling unreferenced (F-23)
Total to interactive (approx.)	~1.29 MB	~365 KB	Before SW caching; repeat visits near-zero

Table 18: First-visit budget, raw and compressed (measured from out/).

What is already fast

The performance story is not one of neglect - most fundamentals are right. Route-level code splitting is genuine (31 lazy views); the service worker makes repeat visits near-zero-transfer; fonts are self-hosted with preloading and no third-party requests; listing images lazy-load with `onError` fallbacks; entrance animations fire once on viewport entry rather than running persistent `rAF` loops; and the skeleton of a Core Web Vitals story exists (server timing headers, hashed immutable assets, preloaded fonts). The measured gaps are concentration problems, not architecture problems: Home is statically imported, pulling `framer-motion` and the heaviest view into the initial chunk; `recharts` doubles; images are unoptimised by format. All three have direct fixes in the Chapter 26 plan.

Finding	Severity	Remediation
recharts duplicated across two async chunks	Medium	Manual shared chunk or single chart module; ~300 KB saved
JPG served where WebP siblings exist	Medium	Swap references; delete unreferenced variants; expect 30-60% image transfer cut
No responsive image handling (srcset/DPR)	Medium	Add srcset from existing variants; set width/height to stop CLS
Home statically imported in shell chunk	Low	Lazy-load Home like other views; shell shrinks by the Home delta
No immutable cache header for <code>/_next/static</code>	Low	Add <code>max-age=31536000</code> immutable rule in <code>vercel.json</code>
5 MB unreferenced imagery + 876 KB PDF-only brand assets	Medium	Delete from <code>public/</code> ; regenerate asset manifest
framer-motion on every property card	Low	Batch or remove card-level motion; add <code>prefers-reduced-motion</code>

Table 19: Performance findings and remediation summary.

Assessment

Scored against the platform's own market context - Android devices on variable data plans - the shipped performance is adequate-to-good, with a clear 25-35 percent improvement available from a week of mechanical work (image formats, shared chart chunk, lazy Home) that requires no architectural change. The audit's one forward-looking caution: performance work is currently unmeasurable in production (no RUM, no analytics, Chapter 21), so every optimisation lands blind and every regression ships unnoticed. The sequencing recommendation is therefore to stand up even a minimal privacy-respecting web-vitals signal in the same sprint as the first optimisation, not after.

17. SEO and Discoverability

The darkest chapter of the audit

If the report had to compress its strategic message into one sentence, it would be this: the platform is invisible to the channel that decides marketplace outcomes. Because the application is a hash-routed SPA whose shell renders a loading fallback in the initial HTML, and because search engines discard URL fragments, **all 87 property pages, every article, and every area guide are unindexable as distinct resources**. A Google query for a Kilimani two-bedroom - the platform's core demand - cannot surface a Keja listing, today or under the current architecture ever. For a listings marketplace competing against BuyRentKenya (the most-trafficked dedicated property portal in East Africa, backed by ROAM Group) and Jiji, both of whom acquire predominantly through organic search, this is not a ranking problem; it is an existence problem.

The metadata that is done well

The audit wants to be precise about what works, because the fix builds on it. The root layout's metadata is professional: a metadataBase anchored to keja.app, a title template, Open Graph with a proper 1200x630 image and en_KE locale, Twitter large-image cards, a robots configuration with max-image-preview large, format detection controls, and application metadata. robots.txt is clean and declares a sitemap. The problem is entirely structural: this excellent metadata describes one page - the home page - and every deep link in the product shares it, because nothing in the SPA mutates document title or description per route, and the sitemap the robots file points at does not exist in production.

The sitemap that serves HTML

A well-built sitemap generator exists in the repository (scripts/generate-sitemap.mjs) - it extracts property IDs, article slugs, and area slugs into roughly ninety URLs - but no workflow and no build command invokes it, so out/ ships without sitemap.xml. The failure mode this creates is worse than a 404: the SPA rewrite serves index.html at /sitemap.xml with HTTP 200, presenting the crawler with a JavaScript loading shell where it expects XML. Wiring the generator into the Vercel build command is a five-minute fix and Chapter 27 schedules it in the first week - but the audit is equally clear that a sitemap of fragment URLs fixes nothing about indexability; it is table stakes for the day path-based URLs exist.

Finding	Severity	Detail and remediation path
Fragment URLs unindexable	Critical	Hash routing discards #/ in crawlers; requires prerendered path routes (Ch. 24 decision framework)
sitemap.xml absent; serves HTML at 200	High	Run generator in build command; add XML content-type route ahead of SPA rewrite
No structured data (JSON-LD)	High	Add Organization, WebSite+SearchAction, and per-listing RealEstateListing blocks - works even pre-prerender on the shell for the site entity
No per-route title/description	High	Add a usePageMeta hook on the custom router; restores social unfurls for shared links immediately
Single-page metadata only	Medium	Consequence of the above; resolved by per-route meta
No hreflang / language signals	Low	Content is English; revisit if Swahili content ships

Table 20: SEO findings register.

The strategic decision

The honest framing the audit offers the platform owner is a choice between two coherent strategies, both legitimate, currently unreconciled. **Strategy A - accept the dark:** lean fully into PWA-first, WhatsApp-first distribution - install prompts, referral links, agent-mediated sharing, and eventually app-store presence via the Capacitor shells. This suits the Kenyan context (WhatsApp is the de facto web), matches the current architecture exactly, and forfeits search permanently. **Strategy B - turn on the lights:** introduce path-based routes with build-time prerendering of the indexable surfaces (listing pages, area guides, articles) while keeping the SPA shell for the app-like flows. Next.js's own guidance ranks static generation first for SEO, and the repository even contains the corpse of the right idea - prerender.mjs, the Vite-era script that rendered route HTML into dist/. Chapter 24 specifies the Strategy B implementation as the audit's recommendation, because a marketplace without an organic acquisition channel is renting its growth from every other channel it uses.

Audit recommendation: adopt Strategy B. Prerender the public catalog (87 listing pages, 21 area guides, the insights corpus) to path-based URLs at build time; keep the hash router or migrate to path routing for the authenticated flows; ship JSON-LD per listing; wire the sitemap. The Auto-Pilot pipeline already rebuilds on inventory changes, so prerendering rides the existing deploy loop for free - new listings become indexable pages within one cron cycle.

Assessment

SEO is the platform's lowest-scoring discipline (1.5 of 5) and simultaneously its highest-leverage opportunity, because the fix is architectural but bounded: the public catalog is small, statically generatable, and already data-driven. The gap between where the platform is (zero organic visibility) and where one sprint of work puts it (fully indexable catalog with structured data) is the largest ratio in this report.

18. Accessibility and User Experience

Accessibility census

For an AI-accelerated solo build, the accessibility floor is remarkably high, and the audit credits it explicitly because it is the discipline most often skipped. The census found: a skip-to-content link implemented correctly (visually hidden until focused); a role=search hero form with an aria-label; 393 aria attributes across the component tree with aria-current on active navigation and tabs; every Lucide icon marked aria-hidden; descriptive alt text including photo-count patterns (title - photo 2 of 5) and empty alt on decorative imagery; label-for pairs on all fourteen fields of the listing wizard; a focus-trap hook with escape handling, focus restoration, and stacked-dialog close ordering; role=status loading states; semantic landmark elements throughout; and Radix primitives supplying the dialog and menu semantics the hand-rolled UI could not be trusted to get right.

Contrast and motion

Two gaps temper the census. First, contrast: the brand gold (#D4B04A) on the cream background (#F7F5EF) measures 1.91:1 - far below the 4.5:1 WCAG AA threshold for body text - and while most of the 148 text-gold usages are decorative (the AI wordmark accent, arrows, icons), the footer tagline and several uppercase micro-labels are real text in real violation. The primary green and white-on-green combinations pass comfortably (4.92:1 and 5.37:1), so the fix is scoped: darken the gold for light-mode text usage only. Second, motion: framer-motion entrance and stagger effects ship with no prefers-reduced-motion strategy beyond a single CSS-level concession - a full reduced-motion media query covering the animation library is a modest, standard addition that respects users with vestibular sensitivity.

Finding	Severity	Remediation
Gold-on-cream text contrast 1.91:1	Medium	Darker gold token for light-mode text; keep gold-on-dark as is (8.12:1)
No prefers-reduced-motion for framer-motion	Low	Global MotionConfig with reduced-motion user respect
Skeletons unused; spinners for loading	Low	Adopt content-shaped skeletons from existing ui/skeleton.tsx
Range-input trust filter untested with SR	Low	Add aria-valuetext labelling; test with TalkBack/VoiceOver
Keyboard gallery operation	Pass	Already implemented
Focus management in dialogs	Pass	useFocusTrap is exemplary

Table 21: Accessibility findings.

User experience inventory

The product's UX surface is broad and internally consistent. Search offers facets (area, type, purpose, bedrooms, price and trust sliders), five sort orders, a compare tray, and saved searches with local alert matching - and the empty state is properly designed (a clear message plus a clear-all affordance) where most SPAs simply render blank. The listing detail page is the platform's strongest surface: gallery with keyboard operation, the trust score breakdown, the Property Passport identity block, verification evidence with freshness chips, an inline mortgage calculator, similar properties, and an agent card whose WhatsApp deep link carries a prefilled listing reference - the one genuinely real conversion path in the product. The design system (shaden on Tailwind 4 tokens, oklch colour space, dark mode via token swap with only 25 explicit dark: overrides) is coherent, and the responsive shell (mobile-first, safe-area insets, a five-tab bottom bar, a four-card grid at wide breakpoints) behaves like a product designed in Nairobi for a phone-first market - because it was.

The conversion gap

The audit's principal UX finding is that **the primary conversion actions are theatrical**: Book a viewing and Request callback render a toast and discard the intent - no capture, no persistence, no notification. In a WhatsApp-first market this is survivable (and the WhatsApp float with prefilled context is the real funnel), but the buttons' apparent functionality overstates their reality, which cuts against the platform's own honesty standard. The remediation ladder is cheap: route both actions through the WhatsApp deep link with intent-specific prefilled text (an afternoon); or capture a lightweight lead form into the existing leads store with an admin-queue surface (a day, no backend needed); or defer to the Phase 2 lead API. The audit recommends the first rung immediately and the second by day 30.

The missing map

One domain-shaped absence stands out in the browsing experience: there is no map. Zero usage of Leaflet, Mapbox, Google Maps, or OSM anywhere in the tree - in a property marketplace whose searchStore even persists stylised area coordinates per saved search. Kenya's renters navigate by neighbourhood and landmark, and every incumbent portal leads its results with a map panel. The audit rates this High for the domain and prescribes the pragmatic version in Chapter 26: a Leaflet (OSM-tiled) results map clustered by area, lazy-loaded behind the results view so the library never touches first load, with the existing per-area coordinates as the data source and per-listing geocoding deferred until a provider is chosen deliberately.

The editorial and content system

A property marketplace lives or dies on the content around its listings, and the platform's editorial layer is more developed than its size suggests. The insights corpus publishes article pages through the same lazy-view pipeline as the product surfaces, with slugged routes (`#/insights/:slug`) and a detail view that carries structured metadata. Twenty-one area guides (`#/areas/:slug`) give each Nairobi neighbourhood a canonical page - the exact asset class that, under Chapter 24's Strategy B prerendering, becomes the platform's SEO beachhead, because area-guide queries ('2 bedroom Kilimani rent') are precisely the demand the incumbents monetise and the audit's keyword logic targets. The claims register, discussed throughout this report as the honesty backbone, is itself an editorial product: a public, machine-readable inventory of every material platform claim, rendered in the Trust Center view - the audit knows of no regional competitor that ships anything comparable.

Asset class	Volume	Strategic role
Property listings	87 (27 seed + 60 auto)	The catalogue; grows via Auto-Pilot cron
Area guides	21 areas	SEO beachhead under Strategy B; local-lore moat
Insights articles	corpus in <code>src/data</code>	Topical authority; share surface for WhatsApp
Claims register	register + Trust Center	Differentiating trust artefact; regulator-facing
Verification evidence	per-listing blocks	The product's core promise, rendered
Documentation suite	18 docs + PDFs	Operational memory; partner credibility

Table 22: Content asset classes and their strategic roles.

The content system's weakness is the same as every other surface's - it is entirely build-time. Articles and guides ship inside the JavaScript bundle as authored data modules, which means updates ride deploys (fine at current cadence), nothing is individually addressable to crawlers (the Chapter 17 finding), and there is no

editorial workflow beyond committing TypeScript. None of that blocks Phase 2, and the authored-in-TypeScript approach has genuine virtues for a solo team (type-checked content, zero CMS attack surface); the audit's single recommendation here is the prerendering dependency already scheduled, which converts this rich content layer from invisible inventory into indexable assets.

Assessment

Accessibility and UX score well above the platform's stage: the foundation is semantic, labelled, keyboard-operable, and visually coherent, with two bounded defects (contrast token, motion preference) and one strategic omission (the map). The conversion-theatre finding is the one that matters most to fix before growth spending, because every unit of traffic driven to a toast-shaped dead end is a unit of trust spent against the platform's core promise.

19. Mobile and Capacitor Strategy

A real strategy, not scaffolding

The mobile story is unusually complete for a web-first platform. The Android directory is a full Gradle project (77 tracked files), not a generated stub: `applicationId com.chacadom.keja`, `minSdk 24` targeting the long Android tail, `targetSdk 36` on current policy, adaptive icons across densities, splash drawables in every bucket, `singleTask` launch mode, and - the detail that proves intent - deep-link intent filters for `https://keja.app` with `autoVerify` plus the `keja://` custom scheme. The iOS project (26 files) carries the Xcode workspace, a Swift `AppDelegate`, the URL scheme in `Info.plist`, and asset catalogs. The Capacitor configuration points `webDir` at the static export itself, so the native shells consume the exact artifact the web deploys - one build, three runtimes, zero drift.

What ships and what is missing

What exists is the packaging layer; what is missing is the release engineering around it. The package scripts (`mobile:sync`, `mobile:android`, `mobile:ios`, `mobile:apk`) and a credible runbook (`docs/MOBILE.md`) cover local builds, but there is no native CI - no workflow builds an APK or archive, despite the deploy pipeline being otherwise strong. There is no signing configuration, no versionCode bump automation (it sits at 1), no store listings, no TestFlight or Play internal-track pipeline, and no crash reporting layer (the shell would host Sentry's native SDK naturally). The deep-link `autoVerify` declaration, notably, requires an `assetlinks.json` served from `keja.app` to actually verify - currently absent, so Android links fall back to the disambiguation dialog.

Capability	Status	Gap to store-ready
Android project	Ready	Signing keystore + versionCode automation
iOS project	Ready	Developer account, provisioning, Podfile via cap sync
Deep links (Android)	Declared	Serve assetlinks.json from keja.app/.well-known to activate autoVerify
Deep links (iOS)	Declared	Associated-domains entitlement at build time
Native CI	Absent	GitHub Actions matrix building debug APK on merge
Distribution	Absent	Play internal track + TestFlight; store listings
Crash reporting	Absent	Sentry native (pairs with web SDK from Ch. 21)
Push notifications	Absent	Requires Phase 2 backend; FCM plugin when it lands

Table 23: Mobile release-engineering readiness.

Strategic value

The audit's judgment is that the Capacitor track is worth completing, and worth completing before it rots: native shell projects decay quickly when unexercised (Gradle and Xcode toolchains move; a year-old untested project is a rebuild). The economics favour it - the marginal cost of an installed Android presence is one signing key, one workflow, and one internal-track submission, on top of an artifact that already exists. The distribution logic favours it too: in a market where app-store presence signals legitimacy to landlords and agents, the store listing is a trust asset as much as an acquisition channel. The one caution is sequencing: push notifications and any native payment flow belong behind the Phase 2 backend, so the first store release should ship as the browse-and-WhatsApp product it currently is, not a simulated one.

Assessment

Mobile readiness is genuine but dormant: 4 of 5 on packaging, 2 of 5 on release engineering. The audit schedules the assetlinks.json activation (which also completes the deep-link story for the PWA's shared URLs) inside the first 30 days, and the signing-and-internal-track work as a day-60 milestone contingent on the Phase 2 decision, since the value of a store presence compounds with real data flows behind it.

20. DevOps, CI/CD and Delivery

Pipeline inventory

Four GitHub Actions workflows govern delivery, and their design is coherent with the static architecture. The deploy pipeline (deploy-vercel.yml) runs on pushes to main: Bun frozen install, the tsc gate, the lint gate, the Vercel CLI prebuilt flow (pull, build with the SW stamp, deploy), and then a live smoke test - five-retry GET on the production URL, verification of the service-worker cache header, the frame-options header, the manifest, and the SPA rewrite on a deep link. The PR pipeline (pr-check.yml) runs the same gates without deploying. The production check (production-check.yml) runs hourly on a matrix - the deployment URL as required, keja.app as warning-only until DNS fully propagates. And the Auto-Pilot (auto-listings.yml) runs four times daily: scan, enrich, dedupe, quality-gate, publish JSON, prettier, commit as the owner identity (the documented Vercel Hobby attribution workaround), push, verify.

What the pipeline gets right

- **Gates that mean something (almost):** typecheck and a real static build on every path to production - the two checks that catch the classes of error this codebase can produce.
- **Post-deploy verification:** the smoke test checks behaviour (rewrite, headers, manifest) rather than mere reachability - rare discipline at this stage.
- **Concurrency control:** deploy groups prevent overlapping production deployments.
- **Toolchain pinning:** Node 22 via nvmrc, Bun 1.3.14 in CI, frozen lockfile installs - reproducible builds in practice, not just in theory.
- **Secrets hygiene:** a single VERCEL_TOKEN remains after the Netlify migration cleanly removed both Netlify secrets; org and project IDs are public non-secrets.
- **Honest failure semantics:** the Auto-Pilot defines exit-code contracts (including a nothing-to-do code) and writes step summaries a human can read later.

Gaps in the delivery system

The gaps cluster around environments and rollback. There is **no staging or preview environment**: every merge to main reaches production, and the PR workflow's build never produces a URL a human could click (a Vercel Hobby-plan consequence of the prebuilt-CLI flow, but also an unexamined choice - GitHub Pages or a second project could serve previews). **Rollback is manual**: the Vercel dashboard offers instant promotion of any prior deployment, and DEPLOYMENT.md documents this, but nothing automates it - the smoke test can fail while the bad deploy stays live until a human notices GitHub's email. **Attribution is a standing compromise**: commits authored as the owner identity to satisfy Hobby-plan rules blur the audit trail between human and machine decisions. And the **unwired sitemap** (Chapter 17) is at root a DevOps gap - a generator that exists but that no pipeline invokes, which is the delivery system's version of a smoke

detector in a drawer.

Capability	Maturity	Notes and next step
Build gates (tsc, lint, build)	Good	Real gates on every path; lint itself neutered by config (Ch. 8)
Deployment automation	Good	CLI prebuilt flow, deterministic; depends on one token
Post-deploy smoke testing	Good	Behavioural checks with retries; extend to sitemap and a listing URL
Monitoring / alerting	Basic	Hourly synthetic check; no alert routing beyond GitHub notifications
Preview environments	Absent	Add a preview target (second project or branch aliasing) before team growth
Rollback automation	Absent	Vercel API promote on smoke-failure is a one-workflow addition
Release management	Absent	No tags, changelogs, or versioning discipline for the app itself
Native CI	Absent	Covered in Ch. 19

Table 24: DevOps maturity assessment.

The Auto-Pilot as a delivery pattern

Worth isolating for a moment: the Auto-Pilot is the repository's most original piece of engineering culture. It treats inventory growth as a delivery problem - commits with quality gates, attributable history, verification jobs, and a promotion path through the same pipeline that ships code - rather than as an admin-UI-and-database problem. For a solo-maintained platform this is the correct trade: the audit trail is the review. Its risks are the ones it documents about itself: synthetic enrichment must never masquerade as verification (enforced by trust caps and claim rules today), and commit volume (34 of the last 38) can drown human history (mitigated by conventional chore prefixes and step summaries). The pattern deserves preservation into Phase 2 - as a feed-ingestion service with the same gates, rather than being discarded in favour of manual listing entry.

Workflow-by-workflow breakdown

Step	Tooling	What it proves (and does not)
Credentials guard	bash	Fails fast with a readable message if VERCEL_TOKEN is absent
Install	Bun 1.3.14, frozen lockfile	Reproducible dependency tree; no drift between runs
Typecheck	tsc --noEmit	The real type gate (compensates ignoreBuildErrors); catches the classes lint cannot
Lint	eslint (config as reviewed in Ch. 8)	Currently weak; passes by construction until re-armed (F-15)
Vercel pull	vercel CLI	Project + environment metadata for the prebuilt flow
Build (prod)	NEXT_STATIC=1 next build + sw-version stamp	Static export plus content-hash SW versioning
Deploy prebuilt	vercel deploy --prebuilt --prod	Artifact promotion; concurrency-group guarded
Smoke test	curl + assertions, 5 retries	Live GET, SW cache header, X-Frame-Options, manifest 200, SPA rewrite on a deep link

Table 25: deploy-vercel.yml - the production path, step by step.

Step	Tooling	Notes
Scan + feeds	zero-dep Node	Simulated scanner and feed adapters; env-passed inputs per injection guidance
Enrich	zero-dep Node	Template enrichment; never claims verification
Dedupe + quality gate	zero-dep Node	Deterministic; exit code 3 = nothing to do
Publish	writes auto-listings.json	Prettier-formatted for reviewable diffs
Commit + push	git as owner identity	Vercel Hobby attribution requirement; triggers deploy
Verify	git + API checks	Step summary written for later human reading

Table 26: auto-listings.yml - the growth loop, step by step (4x daily).

The hourly production check runs the smoke assertions on a matrix - the deployment URL as a required check, keja.app as warning-only while DNS settles - and its alert routing is, today, GitHub's default notification email, which the observability chapter addresses. The PR workflow mirrors the deploy gates without deploying; it has never run against an actual pull request, because none has ever been opened - a

fact that is simultaneously a strength (the mainline has stayed disciplined without review) and a latent risk (the workflow itself is untested in its intended role, and the team has no PR muscle memory for the day it matters).

Assessment

DevOps scores 3.5 of 5 - genuinely strong for a Hobby-tier solo operation, with the deficit concentrated in environments and reversal rather than in building or verifying. The audit's sequencing: wire the sitemap and extend the smoke test (week one, costless); add the rollback-promotion workflow (week two, one file); stand up a preview target when the second contributor arrives; adopt tagging and changelogs when the store-track release work begins.

21. Observability

Production is a black box

The observability finding is categorical: **the platform is blind in production**. There is no error tracking - the global `ErrorBoundary` logs component errors only when `NODE_ENV` is development, so every production client exception vanishes silently. There is no remote product analytics - the analytics module is a deliberate local-only ring buffer (200 events, taxonomy-typed, zero egress) that is admirable as privacy engineering and useless for learning anything about usage off-device. There is no real-user monitoring of web vitals, no funnel measurement, no crash aggregation for the Capacitor shells, no uptime alerting beyond GitHub's default notification for a failed hourly workflow, and no status page. The hourly synthetic check tells the team the origin still returns 200; it cannot tell them that a render regression blanked the properties view on Android 11 at 09:00.

Signal	Coverage	Recommended instrumentation
Client errors	None (dev-only logs)	Sentry browser SDK behind the existing analytics consent posture; source maps uploaded in CI
Web vitals (RUM)	None	web-vitals reporting into the same privacy-respecting collector
Product analytics	Local ring only	Plausible-class (cookieless, KE-hosted or self-host) - preserves the no-tracker property
Uptime	Hourly synthetic	Keep; route alerts to email/WhatsApp beyond GitHub defaults
Deploy health	Smoke test	Extend to sitemap.xml and one deep listing URL
Native crashes	None	Sentry native SDK when store track activates (Ch. 19)
Funnel / conversion	None	Event taxonomy already exists locally; mirror it server-side

Table 27: Observability coverage matrix.

Why the local analytics design helps the fix

The platform's existing analytics module is the right foundation for the remediation, which is worth explaining because it is unusual. The module defines a typed taxonomy of eleven event kinds, respects a consent posture, stores locally, and never transmits. That means the remediation is not add-a-tracker - it is choose-a-destination: a cookieless, script-light collector (the Plausible class, or a self-hosted Countly/Umami behind keja.app itself) can receive the same taxonomy as opt-in beacons, preserving both the privacy promise and the analytic vocabulary the product already speaks. The same one-seam logic applies to errors: Sentry's browser SDK can be initialised inside the existing ErrorBoundary with a release tag stamped from the service-worker version, giving content-addressed error attribution - the version string already encodes the exact artifact hash.

Assessment and sequencing

Observability is the platform's second-lowest score (after testing) and the cheapest gap to close relative to its value: a day of work converts the platform from mute to instrumented. The audit's rule for sequencing is that **no growth activity should precede instrumentation** - paid traffic or partner launches against a blind production system convert spend into unattributable outcomes. Concretely: Sentry (errors) plus a cookieless analytics destination plus smoke-test extension are scheduled inside the first 30 days of Chapter 27, before the SEO prerendering work whose impact they would measure, and before any partner-facing push from the marketing documents.

22. Testing Assessment

Current state: zero

The single most consequential quality finding in this audit is simple to state: the platform ships with no automated tests. The tests directory contains three shell scripts (170 lines) that exercise a .zscripts platform directory which does not exist in the repository - orphaned tooling tests, not application tests. There is no test runner in the dependency tree, no test script in package.json, no configuration, no fixtures, no CI step. Every one of the 106 commits that built this platform - including the 254-file framework rebuild - was verified by typecheck, lint, and build alone.

What existed, and what its loss means

The git archaeology matters here, because the platform once had the right instincts. Before the Next.js rebuild, the repository carried 23 test suites totalling 3,039 lines - landlordStore tests (298 lines), finance tests (160), escalation tests for the AI guard (58), tokenization store tests, verification tests, route tests, SEO meta tests - plus vitest configuration with coverage scripts, a composite verify gate (typecheck, lint with zero warnings, tests, build), husky pre-commit hooks, and prettier. Commit ae0b2e3 deleted all of it alongside the Vite application it tested. The loss is not merely of the tests as artefacts; it is of the **regression memory** they encoded: the finance module's amortisation edge cases, the escalation guard that keeps legal questions out of the AI's confident answers, the store invariants that keep trust scores capped. Documentation still cites one of these suites as evidence (REVIEW_ACTIONS.md references escalation.test.ts as implemented), which makes the gap actively misleading to a reader.

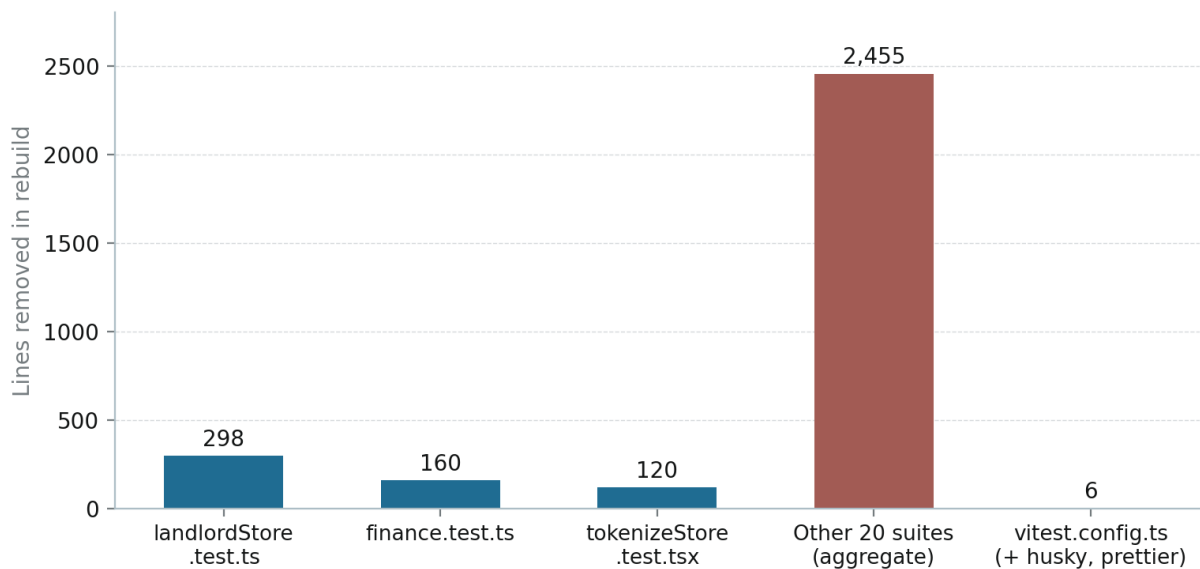


Figure 9: Test coverage deleted in the rebuild commit ae0b2e3, by suite.

Coverage gap analysis

The audit ranked the untested surface by blast radius, on the principle that the first tests restored should guard the code that would hurt most to break silently. At the top sits the **finance module** - pure functions, high consequence, previously tested: amortisation, affordability, and the KES/USD peg arithmetic shown to users in calculators and listings. Second, the **AI engine's escalation guard and intent routing** - the safety property that routes legal, tax, and valuation questions to human handoff, plus the FACT/ESTIMATE labelling that underwrites the platform's trust marketing. Third, the **search matching logic** - the query parser with its subtle K/M/k unit scaling and the trust-floor filters. Fourth, the **tokenization order book invariants** - supply caps, balance conservation, and ledger integrity in the simulation. Fifth, the **store primitive itself** - serialisation, seeding, and event propagation, since every domain depends on it. The views, by contrast, need smoke-level rendering tests first and deep interaction

tests only as Phase 2 lands real flows.

Target	Priority	Rationale and test shape
lib/finance.ts (math)	P0	Pure, high-blast-radius, previously covered; restore original cases from git history and extend
ai/engine.ts (guard)	P0	Safety-relevant escalation and labelling; table-driven intent cases incl. trilingual aliases
searchStore matching	P1	Unit-mixing parser edge cases; property filter invariants
tokenizeStore invariants	P1	Order matching, supply caps, ledger conservation properties
store.ts primitive	P1	Round-trip, seed fallback, event fan-out; jsdom environment
router.tsx navigation	P1	Hash parsing, query params, back/forward
View smoke tests	P2	Render each lazy view without crash; all assertions on the shell
E2E critical path	P2	Playwright: search, open listing, WhatsApp link intent, install prompt suppression

Table 28: Restoration priority: what to test first and why.

Stack recommendation

The 2026-conventional stack for this codebase is Vitest plus React Testing Library for unit and component work (the codebase is ESM, Vite-adjacent, and the deleted suite was vitest-based - the restore path is native), and Playwright for the end-to-end layer (one config can also exercise the installed-PWA context via its persistent contexts, and later the Capacitor web view). The audit recommends restoring before writing: the pre-rebuild suites in git history (git show ae0b2e3^) describe the original invariants in executable form and port to the current modules with modest import changes - a faster route to meaningful coverage than authoring from zero, and one that revives the regression memory rather than approximating it. The composite verify gate should then return to CI exactly as it existed: typecheck, lint, tests, build, in that order, blocking merges.

Implementation sketch

To size the restoration concretely, the audit sketched the target configuration against the codebase's reality: Vitest with a jsdom environment for the store and logic layer, React Testing Library for component behaviour, and Playwright for the end-to-end layer with one project for the hosted SPA and one for the installed-PWA context (persistent context, service worker enabled - the install-prompt and offline behaviours become testable). The vitest configuration is deliberately minimal because the codebase has no path aliases beyond @/, and the deleted pre-rebuild suite ports with import-path changes only.

```
// vitest.config.ts (restoration target)
```

```
import { defineConfig } from 'vitest/config';
import path from 'path';
export default defineConfig({
  test: {
    environment: 'isdom',
    include: ['src/**/*.test.{ts,tsx}'],
    coverage: { reporter: ['text', 'html'],
      include: ['src/lib/**'] },
  },
  resolve: { alias: { '@': path.resolve( __dirname, 'src') } },
});
// package.json scripts (restore the pre-rebuild date)
// "verify": "tsc --noEmit && eslint . --max-warnings 0 && vitest run && next build"
```

The Playwright layer starts with four journeys and grows only as real flows arrive: search-and-open-a-listing (the revenue path), WhatsApp intent handoff (the conversion truth), install-prompt suppression in standalone contexts (the PWA contract), and offline navigation to a cached listing (the platform's differentiating promise). Each journey doubles as a regression net for one Chapter 23 finding family, which is how the audit suggests prioritising when time is scarce: tests that pin the findings, then tests that cover the code.

Assessment

Testing scores 1 of 5 not because the task is large but because it is binary: the safety net is absent, and the platform's most trust-critical code (money arithmetic, an AI that answers questions, a claims register) runs unverified on every deploy. The restoration is tractable - the audit sizes the P0 set at days, not weeks, because the tests once existed and their targets still do. Chapter 27 makes it the third item of the first sprint, immediately after the two security gates, on the principle that everything else this report recommends will be built faster and safer once the net is back.

PART VI - RECOMMENDATIONS

23. Consolidated Findings Register

How to read the register

This chapter consolidates every finding from Parts II through V into a single numbered register, ordered by severity and domain, with the remediation path cross-referenced into Chapters 22 through 24. Severity follows the model defined in Chapter 2: Critical findings expose real users to concrete harm or block a primary channel; High findings will predictably become incidents; Medium findings compound quietly; Low findings are hygiene. In total the audit registers **47 findings: 4 Critical, 13 High, 22 Medium, and 8 Low**. Each row cites its primary evidence location so that any engineer can begin verification without re-deriving the audit's path.

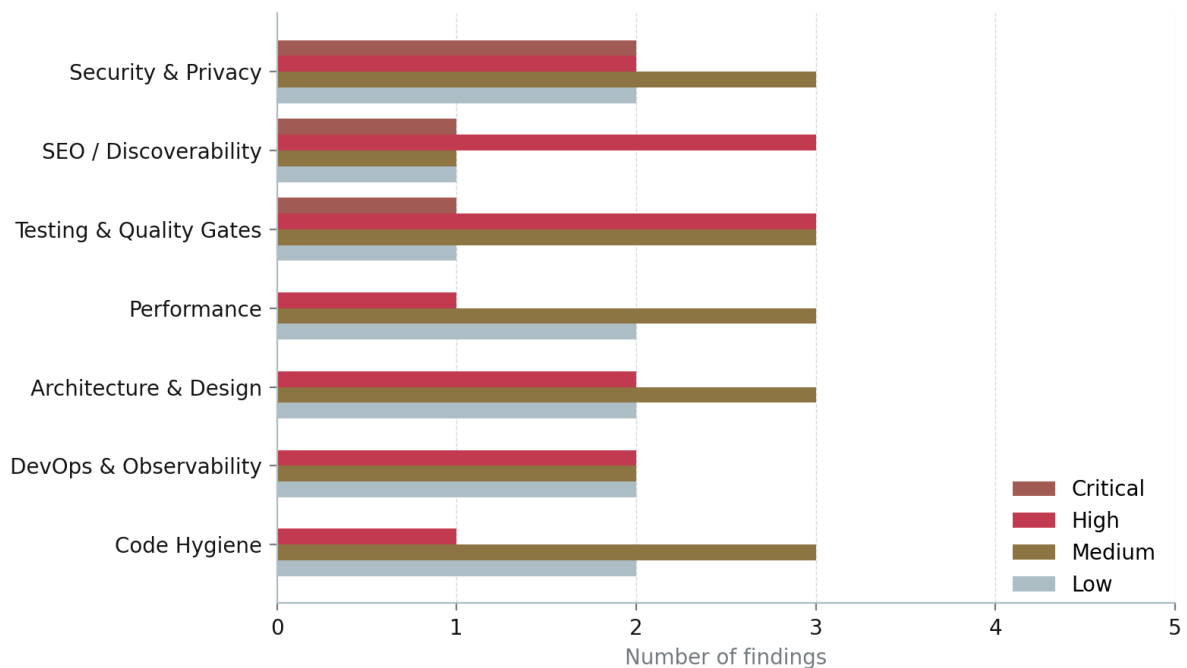


Figure 10: Findings by severity across the seven audit domains.

Critical findings

ID	Finding	Evidence	Remediation
F-01	Passwords hashed with unsalted DJB2 and persisted in localStorage; open registration accepts real credentials	auth.tsx:102-106, 197-202, 395-399	Ch. 24 auth seam; demo-mode gate in <= 5 days
F-02	KYC collects national ID and source-of-funds into plaintext client storage without notice or safeguards	tokenizeStore.tsx:188-196, 246-252; TokenizeView.tsx:310-339	Ch. 14 gating; ODPC registration path; encrypt server-side
F-03	All property, article, and area pages unindexable by search engines (hash-routed CSR shell)	router.tsx; out/index.html initial payload	Ch. 24 Strategy B prerendering decision
F-04	Zero automated tests; 3,039 lines of suites deleted in rebuild commit	tests/ orphan scripts; git show ae0b2e3	Ch. 22 restoration plan; verify gate in CI

Table 29: Critical findings (activate with real users or block a primary channel).

High findings

ID	Finding	Evidence	Remediation
F-05	#/admin console renders with no authentication or role check	KejaApp.tsx:121; AdminView.tsx	Gate in UI now; server-side check in Phase 2
F-06	No Content-Security-Policy despite otherwise complete header set	vercel.json:42-50	Deploy default-src 'self' (report-only first)
F-07	sitemap.xml absent while robots.txt declares it; serves HTML at 200	scripts/generate-sitemap.mjs unwired	Wire into build command; content-type route
F-08	No structured data of any kind (Organization, RealEstateListing, Breadcrumbs)	out/index.html grep	JSON-LD blocks; works pre-prerender
F-09	No per-route title/meta; every share unfurls as home page	No document.title mutation in src/	usePageMeta hook on the router
F-10	No production error tracking; boundary logs dev-only	ErrorBoundary.tsx:21-24	Sentry init; CI source maps
F-11	No remote analytics or RUM; product learning impossible off-device	analytics.ts local ring	Cookieless collector; mirror taxonomy
F-12	No map UI in a property marketplace	0 hits: Leaflet/Mapbox/Google/OSM	Ch. 26 Leaflet clustered results map
F-13	recharts duplicated across two async chunks (~300 KB)	out/_next/static/chunks sizes	Shared chunk or single chart module
F-14	Dead code mass: 30 shadcn files (4,243 LOC), db.ts, ~20 unused deps	Import graph analysis	Ch. 26 purge; lockfile shrinks accordingly
F-15	Lint configuration disables 26 rules incl. correctness set	eslint.config.mjs	Re-arm incrementally; zero-warnings gate
F-16	No staging/preview environment; every merge is production	Workflows; Vercel Hobby	Preview target; PR builds to URL
F-17	Deep-link autoVerify unactivated (no assetlinks.json served)	android/ intent filters; public/ absent	Serve .well-known/assetlinks.json

Table 30: High findings (predictable future incidents).

Medium findings

ID	Finding	Remediation
F-18	noImplicitAny: false; ignoreBuildErrors; reactStrictMode off	Re-arm config after P0 fixes
F-19	Unvalidated JSON.parse of persisted state on every read	Zod schemas at the read boundary
F-20	auto-listings.json trusted via as-unknown-as cast (5,630 lines)	Validate generated data at boundary
F-21	Cross-tab coherence only for auth; useStore event gap	Move storage listener into store primitive
F-22	Monolithic views (ManageView 523-line function; DealAnalyst 418)	Extract pure logic modules first
F-23	JPG served where WebP siblings exist; ~5 MB dead imagery deployed	Reference swap + public/ diet
F-24	No responsive images (srcset/DPR); layout shift on load	srcset from existing variants
F-25	Gold-on-cream text contrast 1.91:1 on real text	Darker gold token for light mode
F-26	Book a viewing / Request callback are toast stubs	WhatsApp intent deep links; then lead store
F-27	i18n claimed trilingual; ~20 keys wired across 6 components only	Complete or de-claim the story
F-28	Offline page uses retired gold palette	Rebrand offline.html
F-29	No prefers-reduced-motion for framer-motion	Global MotionConfig
F-30	Prisma scaffold implies a database that does not exist	Author real schema (Ch. 24) or delete scaffold
F-31	Caddyfile arbitrary-port proxy pattern tracked in repo	Delete or bind localhost with allowlist
F-32	No immutable cache header for /_next/static	One vercel.json rule
F-33	No native CI or signing for Capacitor projects	Ch. 19 release engineering
F-34	No rollback automation on smoke failure	Vercel API promote workflow
F-35	Seed PII in real-contact format across auth and stores	Reserved-domain fictional data
F-36	Silent service-worker updates (no version toast)	Toast with reload affordance
F-37	Date.now() used as identifier at 30+ sites	Central uuid helper
F-38	Orientation lock blocks landscape/tablet browsing	Remove lock; test landscape
F-39	Tailwind v3 config file contradicts v4 CSS-first reality	Delete tailwind.config.ts

Table 31: Medium findings (compounding costs).

Low findings

The eight Low findings are hygiene items, listed for completeness and scheduled into the Chapter 26 excellence plan rather than the critical path: the two dev-only console statements (both intentional); the thirteen guarded non-null assertions in the compare feature (a narrowing helper would remove the class); the void-marker dead statements in two views; the duplicated local format helpers that should consolidate onto `src/lib/format.ts`; the unused skeleton component while loading states use spinners; the untested range-input trust filter with screen readers; the hardcoded org and project IDs in the deploy workflow (public values, but vars are cleaner); and the missing `.env.example` for the Next.js era. None of these blocks anything; all of them are the difference between a repository that feels maintained and one that is.

24. Phase 2 Target Architecture

The one-seam principle

The audit's architectural recommendation is deliberately conservative: **change one thing, not everything**. The platform keeps its static SPA shell, its hash-to-path routing evolution, its PWA and Capacitor surfaces, and its entire component layer. It gains exactly one new element - a typed API layer - behind which live the four services the product narrative already promises: PostgreSQL with a real Prisma schema, server-side authentication, M-Pesa integration through a licensed payment service provider, and search. Every Phase 2 migration then proceeds one domain at a time through the store boundary that Chapter 10 identified as the codebase's natural seam. This is the difference between an evolution the current codebase was accidentally designed for and a rewrite nobody asked for.

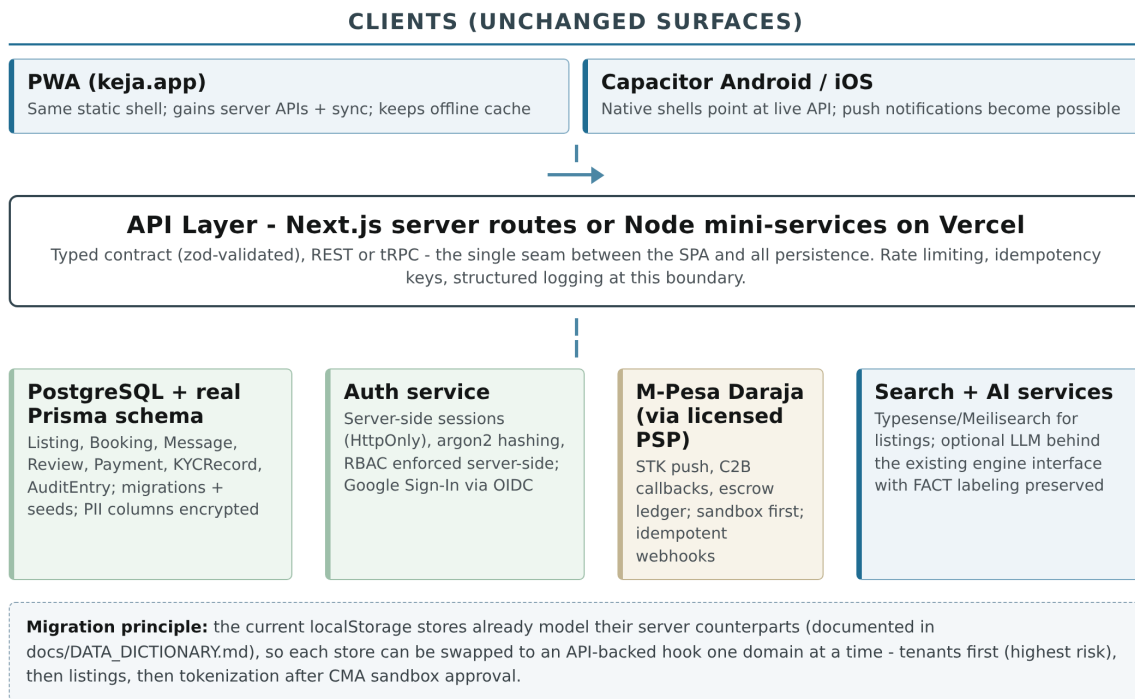


Figure 11: Phase 2 target architecture: the client surfaces stay; one typed API seam adds persistence, auth, payments, and search.

The API seam

The seam should be boring and typed. Next.js server routes (the same repository, the same deploy) or a small Node service on Vercel expose a REST-or-tRPC contract validated with zod at the edge - the same zod that sits unused in package.json today, finally earning its place. Three properties are non-negotiable at this boundary: idempotency keys on every mutation the client might retry (payments above all), rate limiting per identity and per IP, and structured request logging with the release tag inherited from the service-worker version stamp. The client-side migration mechanism is equally specific: each domain store gains an API-backed twin behind its existing hook signature, with localStorage retained as the offline cache - the store that was built to work without a server becomes the offline-first cache that makes the server safe to add. Tenants migrate first (highest risk, smallest surface), then listings and moderation, then landlord ledgers once reconciliation exists, and tokenization last, behind CMA sandbox admission.

The data layer

Chapter 13 specified the schema shape; the architecture adds its operational properties. PostgreSQL (managed - Neon, Supabase, or RDS-class) with the authored Prisma schema as the single source of truth; migrations from day one rather than db:push against a moving database; seeds derived from the existing data dictionary so the demo experience survives the migration; PII columns (identifiers, income, contact) encrypted at rest or vaulted; and the audit-entry model promoted to a first-class table so the platform's transparency property survives contact with a database. The Auto-Pilot pipeline's Phase 2 form is a feed-ingestion service that writes through the same quality gates into the listings table - the growth loop preserved, now with real storage behind it.

Authentication

The auth migration is the highest-risk, highest-leverage seam, and the audit specifies it precisely. Server-side sessions (HttpOnly, Secure, SameSite=Lax cookies) issued by an OIDC-capable provider - the existing optional Google Sign-In already speaks the right protocol - with argon2id password hashing server-side, and the demo accounts rebuilt as labelled demo identities that cannot be confused with real users. The RBAC model (user, agent, admin) moves from a client-side boolean to a server-side authorization check on every admin-scoped endpoint; the UI gate from finding F-05 becomes the client half of a defense in depth. The existing 12-hour sliding session semantics carry over. Nothing about this requires NextAuth specifically (the unused v4 dependency should not be revived); a thin session service or a current-generation library both fit the seam.

Payments: M-Pesa done correctly

The Kenyan payment rail is M-Pesa, and the audit's recommendation follows the platform's own documented caution: integrate Daraja (Safaricom's API) through a licensed PSP arrangement rather than as an independent merchant until licensing is secured - current integration guidance puts the full merchant path at roughly five weeks including business registration, paybill/till provisioning, and KRA documentation. The technical shape is well understood and the codebase's ledger already models it: STK push for payment initiation, C2B callbacks for confirmation, a double-entry escrow ledger with hold, release, and refund semantics (B2C plus reversal), idempotency keys on every webhook, and reconciliation against Safaricom's transaction reports. The sandbox comes first, hard-wired behind a feature flag, with the simulation retained as the demo path - the same honesty pattern the platform already applies to verification.

The SEO decision, implemented

Chapter 17 committed to Strategy B; this chapter specifies it. The public catalog - listing pages, area guides, the insights corpus, the claims register - becomes build-time prerendered HTML at path-based URLs (Next.js generateStaticParams over the inventory, or the modernised prerender script applied to out/). The SPA shell continues to own the interactive flows (search with facets, dashboards, applications) under whatever routing serves them best, with per-route metadata from the usePageMeta hook regardless. Each listing page carries JSON-LD (RealEstateListing plus BreadcrumbList), the sitemap generator runs in the build command, and the Auto-Pilot deploy cycle guarantees freshness: a listing ingested at the 06:00 cron run is an indexable URL by 06:20. The one migration cost the audit flags honestly: shared fragment links in the wild (WhatsApp history, saved posts) need a one-time redirect map from hash routes to path routes in the shell.

API contract sketch

To make the seam concrete enough to estimate, the audit sketched the initial endpoint surface - deliberately small, resource-shaped, and versioned from day one. The contract follows three conventions: every mutation accepts an Idempotency-Key header; every response carries the artifact release tag; and errors use a closed taxonomy (validation, auth, rate-limit, conflict, internal) so the client can render honest messages rather than raw failures. The first wave totals eleven endpoints - small enough to review in one sitting, which is the point.

Endpoint	Method	Purpose	Notes
/v1/auth/session	POST	Email + password exchange for HttpOnly session	argon2 verify; logout counters server-side
/v1/auth/session	DELETE	Sign out	Revokes server session
/v1/auth/oidc/google	POST	OIDC code exchange	Existing optional flow, promoted
/v1/auth/demo	POST	Labelled demo identities	Cannot mint roles; rate-limited
/v1/me	GET	Session user + profile	Drives all client gates
/v1/me	PATCH	Update profile, language	zod-validated
/v1/properties	GET	Public catalog (paged, filtered)	Feeds search + prerender + PWA cache
/v1/properties/:slug	GET	Listing detail with evidence	JSON-LD mirrors this shape
/v1/applications	POST	Submit tenancy application	Idempotency-Key required; PII columns encrypted
/v1/leads	POST	WhatsApp-ref or form lead capture	Closes finding F-26
/v1/viewings	POST	Book a viewing	Notifies landlord channel

Table 32: Wave 1-2 endpoint surface (auth, profiles, applications, leads).

```
// Store-twin migration sketch (tenant domain)
// Before (today): const [app, setApp] = useTenantStore();
// After:          same hook. API-backed twin. localStorage as cache
async function submitApplication(draft, idemKey) {
  cache.set('keia:tenant:pending', draft); // offline-first
  const res = await api.post('/v1/applications', draft,
    { headers: { 'Idempotency-Key': idemKey } });
  cache.delete('keia:tenant:pending');
  return res.data; // queue drains on reconnect
}
```

Sequencing

The architecture lands in four moves, each independently shippable. Move one is foundational hygiene: the security gates, the test restoration, the sitemap and metadata - all Chapter 27 P0 items that Phase 2 code should not precede. Move two is the seam itself: schema authored, API skeleton with auth and the tenant domain, preview environment live. Move three is the growth surface: prerendering, JSON-LD, lead capture real, map shipped, analytics instrumented. Move four is transactions: M-Pesa sandbox behind the PSP track, landlord ledgers migrated, store releases following. Each move has a rollback that is simply the previous state - the architecture never bets the platform on a single cutover.

25. Phase 2 Data Schema Blueprint

From dictionary to schema

This chapter operationalises Chapter 13's gap analysis into the concrete model list the audit recommends authoring as the first Phase 2 commit. Every model below traces to interfaces that already exist in the client stores and the data dictionary - the schema is a translation, not an invention - and every constraint listed is one the client currently enforces by discipline alone. The blueprint is deliberately conservative: it models exactly what the product ships today plus the two Phase 2 additions the roadmap requires (payment records and sessions), and defers custody and settlement tables to the tokenization track where the CMA sandbox will dictate their shape.

Model	Key fields and constraints	Traces to / notes
User	id, email unique, passwordHash?, role enum, status, timestamps	auth.tsx User; adds server-side hash and status
Session	token (opaque), userId FK cascade, expiresAt, createdAt	12h/30d sliding semantics carried over
Profile	userId unique FK, name, phone, avatarUrl, language enum	keja:profile; language from i18n
Property	id, slug unique, title, type enum, purpose enum, areaId FK, priceKes, bedrooms, description, status enum, source enum (seed/auto/user), timestamps	properties.ts Property; source distinguishes provenance
PropertyEvidence	id, propertyId FK cascade, kind enum, label, detail jsonb, verifiedAt, expiresAt, method enum	verification blocks; method never 'simulated' for real rows
Area	id, slug unique, name, aliases text[], description	neighborhoods.ts + engine alias tables
TenancyApplication	id, propertyId FK, userId FK, status enum, monthlyIncomeKes, employer, references jsonb, createdAt	tenantStore RentalApplication; PII - encrypt columns
Viewing	id, propertyId FK, userId FK, scheduledFor, status enum, notes	Book-a-viewing intent (F-26); new
Lead	id, propertyId FK, name, phone, message, source enum, createdAt	keja:leads; WhatsApp-ref or form
RentPayment	id, unitId FK, tenantId FK, period (unique with unitId), amountKes, channel enum, reference, paidAt	landlordStore RentPayment; uniqueness the client could not enforce
Unit	id, propertyId FK cascade, label, rentKes, status enum	landlordStore units
TenantRecord	id, unitId FK, userId?, name, phone, startedAt, endedAt?	landlordStore tenants
MaintenanceTicket	id, unitId FK, title, body, priority enum, status enum, createdAt	landlordStore tickets
AuditEntry	id, actorId?, action enum, targetType, targetId, meta jsonb, createdAt	adminStore audit trail (bounded 500 client-side)
ListingSubmission	id, submitterId?, payload jsonb, status enum, reviewerNote, createdAt	keja:submissions moderation queue
SavedSearch	id, userId FK, criteria jsonb, alertsEnabled, createdAt	searchStore; alerts run server-side
KycRecord	id, userId FK unique, status enum, idType enum, idNumberEncrypted, sourceOfFunds, decidedAt	tokenizeStore; encrypted at rest (F-02)

Table 33: Core domain models for the Phase 2 schema (traced to client contracts).

Enums and integrity rules

The schema's enums encode the TypeScript unions that currently exist only at the edges: PropertyType, Purpose, Availability, ClaimStatus, SubmissionState, Role, PaymentChannel (mpesa, bank, cash - exactly the client's model), TicketPriority, KycStatus, and EvidenceKind. The integrity rules the database adds beyond the client discipline: unique (unitId, period) on RentPayment so double-entry is impossible; unique (userId) on KycRecord so identity cannot fork; cascade deletes owned through the property graph (evidence, units, submissions) while financial records (payments, applications) restrict deletion and soft-delete instead; and an audit trigger on status transitions for Submission and KycRecord rows so the moderation trail is write-once. The audit's one schema-level opinion beyond translation: keep the claims register as a **materialised table** (Claim rows with lastVerifiedAt) rather than a static data file, so the Trust Center view the brand depends on becomes queryable history the moment Phase 2 lands.

Migration waves

Wave	Domains	Depends on	Exit criteria
1	Auth, Sessions, Profiles	Schema + API skeleton	Server-issued HttpOnly sessions; demo identities labelled
2	Tenancy applications, Leads, Viewings	Wave 1	Application crosses devices; landlord notified
3	Properties, Evidence, Areas, Submissions	Wave 1; moderation queue	Auto-Pilot writes via service; sitemap from DB
4	Units, Tenants, Payments, Tickets	Waves 1-3; M-Pesa sandbox	Ledger reconciled against Daraja callbacks
5	KYC, tokenization (deferred)	CMA sandbox track; ODPC registration	Regulated-path custody decision made

Table 34: Domain migration waves through the store-twin seam.

Each wave ships behind its store twin with the localStorage path retained as fallback for one release, then removed; the offline cache story from Chapter 10 makes the retention period a product decision rather than a technical one. The blueprint's final property is the one the audit most wants preserved from Phase 1: because every model traces to a documented client contract, the data dictionary remains true documentation of the platform's state - now with a database that enforces it.

26. Engineering Excellence Plan

Guardrails back on

The excellence plan begins where Chapter 8 ended: restoring the configuration the code's actual quality already earns. The sequence is incremental so nothing breaks at once. First, set `noImplicitAny` true and remove `ignoreBuildErrors` from `next.config` (CI already runs `tsc`, so the local-build risk is theatre anyway); fix whatever the compiler finds - the audit expects little, given zero explicit-any usage. Second, re-arm the ESLint correctness rules (`no-undef`, `no-unreachable`, `no-fallthrough`, `no-redeclare`) immediately and the hygiene rules (`no-explicit-any`, `no-unused-vars`, `prefer-const`) over the following month as warnings-then-errors. Third, restore the composite verify gate from the pre-rebuild era - typecheck, lint with zero warnings, tests, build - as the merge condition, enforced by the PR workflow that has never yet run on a real PR. Fourth, re-enable `reactStrictMode` and fix the double-render assumptions it surfaces while the codebase is young enough for that to be cheap.

The purge

Subtraction is the highest-leverage week available. Delete the 30 never-imported `shadcn` files (4,243 lines) and the four dependencies that exist only for them (`embla-carousel`, `react-day-picker`, `react-resizable-panels`, and the `react-hook-form` stack). Delete `src/lib/db.ts`, the Prisma scaffold (or replace it with the authored schema, per Chapter 24 - the audit's preference), and the ~16 remaining unused dependencies (`zustand`, `next-auth`, both `TanStack` packages until the seam needs them, `zod` until the boundary validation lands, `dnd-kit`, `next-intl`, `date-fns`, `uuid` - re-adding it when the `id` helper lands - `react-syntax-highlighter`, the MDX editor, `@reactuses/core`, `z-ai-web-dev-sdk`). Delete the dead `tailwind.config.ts`, the orphaned `tests/*.sh` scripts, and the Vite-era `prerender.mjs` in favour of its modernised successor. The measurable outcomes: a lockfile that shrinks by hundreds of packages, CI installs that speed up, a security-scanner surface that reflects reality, and a `package.json` that stops telling newcomers lies about the stack.

Refactoring with intent

The refactoring agenda is narrow and test-driven - each extraction lands alongside the tests it enables. `ManageView`'s 523-line closure splits into a pure `landlord-metrics` module, an `alert-rule` evaluator, and a thinner view; `DealAnalyst` follows the same cut; the AI engine's `respond` routine separates its intent tables from its decision logic so the escalation guard becomes directly testable. The store primitive absorbs the cross-tab listener currently living in `auth.tsx`, and its fallback-dependency hazard is fixed with a `ref-stable` default pattern. The auto-listings JSON gains a `zod` validation step at the read boundary. None of this is cosmetic restructuring; every extraction is chosen because Chapter 22's test priorities need the seam it creates.

Craft standards to adopt

- **One uuid helper** (crypto.randomUUID) replacing 30+ Date.now() identifier sites - one module, one grep, done in an hour.
- **Format consolidation:** every money/percentage formatter lives in src/lib/format.ts; the ten local variants delete.
- **Reduced motion:** a global MotionConfig respects prefers-reduced-motion across the framer-motion surface.
- **Contrast token:** a light-mode-safe gold variant for text usage; the WCAG check joins the verify gate as an axe pass on the shell.
- **Commit conventions:** conventional prefixes stay; add a lightweight PR norm (even self-review) from the first second contributor, and move workflow constants to repo variables.
- **Docs currency:** REVIEW_ACTIONS.md updates to reflect the deleted-test reality; REPO_PICTURE.md's live-URL claim re-points at the verified production domain.

What deliberately stays

Excellence also means refusing fashionable churn. The audit recommends **against:** migrating the custom router to react-router before the prerendering decision constrains it properly; adopting react-query wholesale before there is an API to query (the store seam will pick its own server-state library when it needs one); rewriting the AI engine as an LLM integration before its regression tests exist and its guard behaviour is pinned; and any move off the static-first shell that does not come through the Chapter 24 seam. The platform's best engineering qualities - coherence, honesty, uniformity - are exactly what churn erodes first.

27. 90-Day Action Plan

Sequencing logic

The plan sequences the register by dependency, not merely by severity: security gates land before growth spending, instrumentation lands before the optimisations it would measure, the safety net lands before the refactors that need it, and every Phase 2 structural item follows the hygiene it depends on. P0 items belong to the first fortnight; P1 items to the first month; P2 items to the quarter. Nothing in the plan requires more than one engineer, and the totals are honest: the audit estimates the P0 set at one focused week, the P1 set at two to three weeks, and the P2 set as the beginning of the Phase 2 proper rather than its completion.

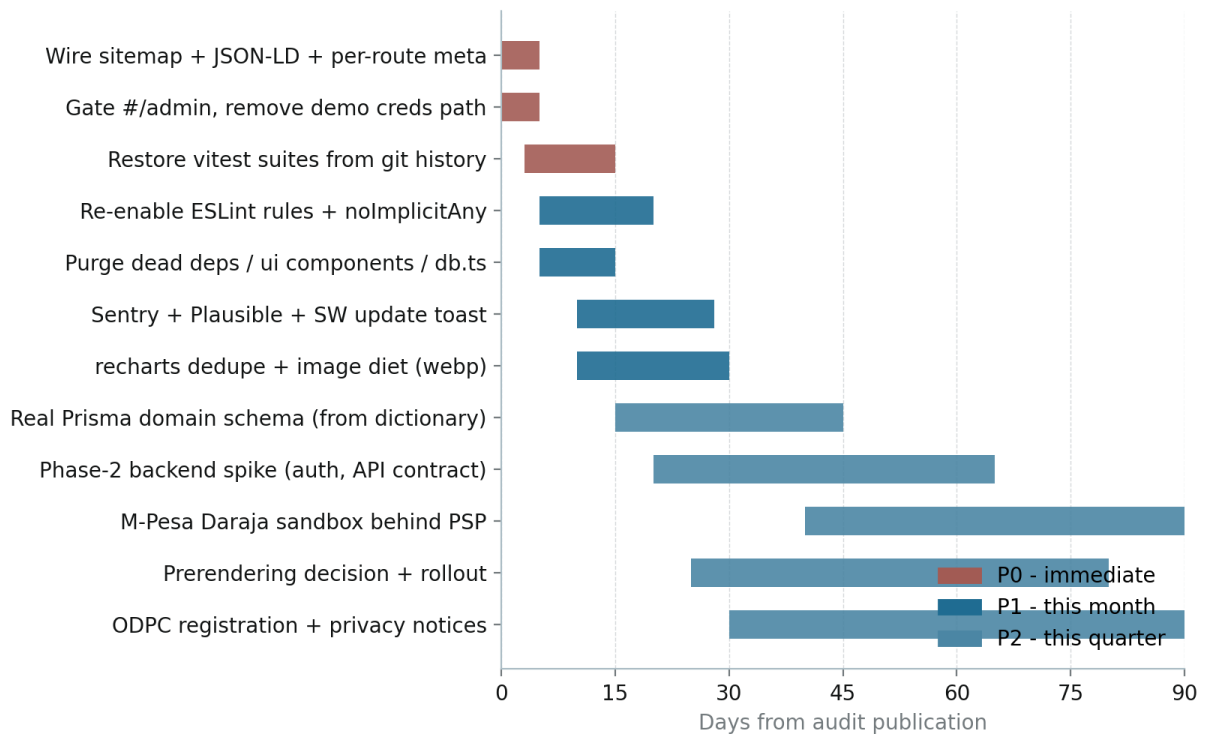


Figure 12: The 90-day action plan: twelve workstreams sequenced by dependency, coloured by priority.

Ref	Action	Effort	Definition of done
P0-1	Gate #/admin behind isAdmin; add demo-credential warning to real registrations	0.5 day	Console renders sign-in gate for anonymous visitors
P0-2	Stop persisting KYC identifiers in demo; add collection notices	0.5 day	KYC form discards or blocks identifier persistence; notice text present
P0-3	Wire generate-sitemap.mjs into build; XML content-type ahead of SPA rewrite	0.5 day	keja.app/sitemap.xml returns valid XML, 200
P0-4	Add usePageMeta hook; per-route titles and descriptions	1 day	Deep-link shares unfurl with listing titles
P0-5	Restore P0 test set from git history (finance, escalation guard) + verify gate in CI	2-3 days	vitest green in CI; merge blocked without it
P0-6	Sentry browser init in ErrorBoundary; release tag from SW version	0.5 day	Test error visible in Sentry with artifact hash
P0-7	Deploy CSP report-only, then enforce default-src 'self'	0.5 day	Header live; report endpoint quiet under normal use

Table 35: P0 - first fortnight (safety and truth).

Ref	Action	Effort	Definition of done
P1-1	Re-arm noImplicitAny, ignoreBuildErrors off, ESLint correctness rules	1 day	Verify gate passes with strict flags
P1-2	Dead-code purge: 30 ui files, db.ts, ~20 deps, dead configs	1 day	Lockfile shrinks; imports unchanged; build green
P1-3	Image diet: WebP references, delete unreferenced assets, srcset	2 days	Listing card transfer drops 30%+
P1-4	recharts shared chunk; lazy Home	1 day	Lazy chunks no longer duplicate charting lib
P1-5	Cookieless analytics destination mirroring local taxonomy	1 day	Events visible off-device; no cookies set
P1-6	Extend P1 test set: search matching, tokenize invariants, store primitive	3 days	Critical-logic coverage restored
P1-7	Conversion honesty: WhatsApp intent links on both stub CTAs	0.5 day	Every CTA either captures or hands off truthfully
P1-8	Rollback-promotion workflow on smoke failure; smoke covers sitemap + deep listing	1 day	Auto-revert observable in a drill
P1-9	assetlinks.json for Android deep-link verification; offline page rebrand; orientation unlock	1 day	Links open the app directly; landscape usable

Table 36: P1 - first month (excellence and instrumentation).

Ref	Action	Effort	Definition of done
P2-1	Author real Prisma schema from data dictionary; migrations + seeds	1 week	Schema reviewable; demo data seeded from dictionary
P2-2	API seam skeleton: auth service + tenant domain behind store twins	2 weeks	Registration server-side; tenant app crosses devices
P2-3	Prerender public catalog at path URLs; hash-to-path redirect map; JSON-LD	2 weeks	Listing pages indexable; Search Console verifies
P2-4	Leaflet clustered results map, lazy-loaded	3 days	Map panel on results; no first-load cost
P2-5	M-Pesa Daraja sandbox behind PSP track; escrow ledger model	2-3 weeks	Sandbox STK push round-trips; idempotent webhooks
P2-6	ODPC registration; privacy notices live at collection points	1 week + processing	Registration certificate; notices shipped
P2-7	Preview environment; native debug-APK CI; store-track preparation	1 week	PRs get URLs; APK artifact per merge
P2-8	Monolith refactors driven by restored tests (Manage, DealAnalyst, engine)	ongoing	Pure modules extracted; views thin

Table 37: P2 - the quarter (Phase 2 proper begins).

Effort and impact matrix

For a small team deciding where an afternoon goes, the matrix below ranks the highest-leverage actions from the register by effort against expected impact. Its shape is the plan's thesis in miniature: the upper-left quadrant (high impact, low effort) is crowded - a genuinely fortunate situation that will not persist once Phase 2 begins - while the expensive items are few and mostly structural. Working the quadrant in order is the audit's final, practical advice.

Action	Effort	Impact	Why it ranks here
Wire sitemap into build	Hours	High	Closes a High finding; prerequisite for any SEO work
Gate #/admin + KYC demo mode	Hours	High	Defuses both Critical security exposures' near-term risk
usePageMeta per-route metadata	Day	High	Every shared link unfurls correctly; cheap credibility
Sentry + release tags	Hours	High	Converts production from mute to observable
Dead-code and dependency purge	Day	Medium	Faster CI, honest package.json, smaller audit surface
Restore P0 test suites + verify gate	Days	High	The safety net everything else depends on
Image diet (WebP + srcset)	Days	Medium	30%+ transfer cut on every listing-bearing page
Prerender public catalog	Weeks	Critical	Opens the organic acquisition channel (Strategy B)
API seam + auth migration	Weeks	Critical	The Phase 2 unlock; every transactional feature waits on it
M-Pesa sandbox behind PSP	Weeks	High	Revenue rail; gated on licensing as much as code

Table 38: Highest-leverage actions by effort and impact.

Cadence and verification

The plan's verification cadence is weekly and cheap: the verify gate must stay green (it becomes the heartbeat), the production check gains the sitemap and deep-listing assertions, Sentry's release health becomes the morning read, and the findings register in Chapter 23 is re-scored at day 30, 60, and 90 with every closed item cited to its commit. The audit's definition of success for the 90 days is precise: all four Critical findings closed, all thirteen High findings closed or in P2 flight, the test count above the pre-rebuild watermark, and the first Phase 2 seam (authentication plus tenant applications) live behind a preview environment - at which point this document's successor should be a much shorter health check rather than another hundred-page recovery plan.

28. Risk Register

How the register was built

The audit closed by translating its findings into forward-looking risks - events, not conditions - scored on likelihood and business impact, each with a mitigation already present in this report's recommendations. Likelihood reflects the audit's judgment of probability over the next two quarters if no remediation occurs; impact reflects consequence to the platform's users, reputation, or viability. The ten risks below are the ones that move: the long tail of Low findings is deliberately excluded, and every risk maps to at least one action in the Chapter 27 plan so that closing the plan closes the register.

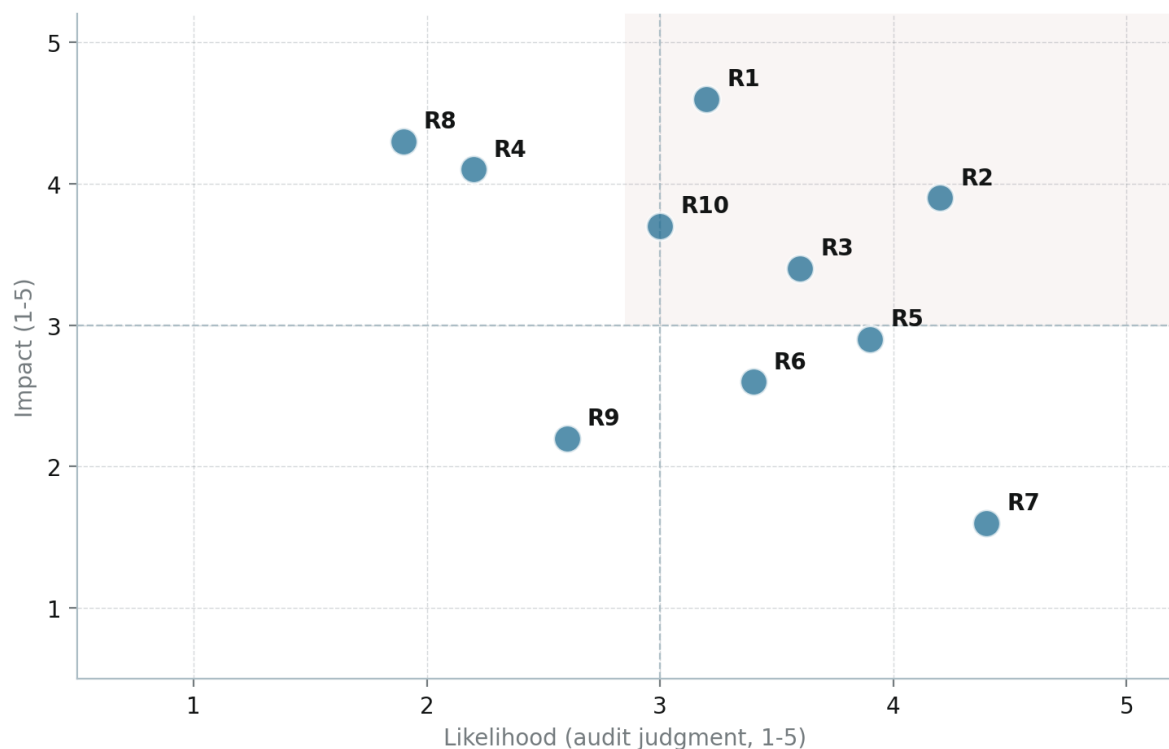


Figure 13: Risk matrix: likelihood versus impact, with R1-R10 keyed to the register below.

ID	Risk	Driver (evidence)	Mitigation (plan refs)
R1	A real user's real password is exposed through client-side storage	F-01; open registration + DJB2 + reuse norms	P0-1 demo gating; P2-2 server auth
R2	Organic acquisition stays at zero while spend substitutes for search	F-03/F-07/F-08; hash routing	P0-3/P0-4 metadata; P2-3 prerendering
R3	A silent regression in money math ships to users	F-04; zero tests on finance module	P0-5 restored suites; verify gate
R4	Regulatory action on KYC collection without registration	F-02; DPA 2019 obligations	P0-2 gating; P2-6 ODPC path
R5	Platform hits the multi-device wall mid-growth (landlord-tenant data never meets)	Architecture ceiling (Ch. 5, Ch. 10)	P2-2 seam; domain-by-domain migration
R6	Production incidents stay invisible until users report them	F-10/F-11; no telemetry	P0-6 Sentry; P1-5 analytics
R7	Dead code and disabled gates erode contributor trust and onboarding	F-14/F-15; scaffold inheritance	P1-1/P1-2 purge and re-arming
R8	Tokenization marketing outruns CMA sandbox reality	Ch. 14 sequencing; claims register honesty	Keep claims register current; gate KYC (P0-2)
R9	A supply-chain incident rides the oversized dependency tree	1,022 packages; unused risky deps	P1-2 purge; CSP (P0-7) as blast limiter
R10	Single-maintainer concentration halts delivery	Solo history; 8 author identities, 1 human	Docs, tests, runbooks (this report); PR norms at second contributor

Table 39: The risk register with mitigations mapped to the action plan.

Closing statement

Audits are easy to write as indictments and hard to write as instruments, and the difference is evidence. This one has tried to hold its standard to what the repository actually contains: a platform that in thirteen days went from an idea to a deployed, installable, monitored, self-growing marketplace with the best honesty infrastructure in its regional class - and that carries, on the other side of the ledger, four critical omissions whose fixes are all known, sized, and sequenced. The audit team's parting judgment is deliberately plain: the codebase deserves the Phase 2 it is being prepared for. The store layer was built with its server twin in mind; the claims register was built to survive regulatory scrutiny; the service worker was built to survive Kenyan connectivity; and the documentation was built to survive its author. Those are the properties that compound. Execute Chapter 27's first month, and the platform's next audit should read as a

health check - shorter, duller, and considerably more profitable.

The register's shape

Read as a shape rather than a list, the register says something encouraging: the platform's worst risks are not scattered - they concentrate in the gap between what the product claims and what the architecture currently delivers (R1, R4, R5, R8), and that gap is precisely what the Chapter 24 seam closes. The second cluster is blindness (R3, R6) - closed by a week of instrumentation and restored tests. Only R10, the human factor, has no technical mitigation; it is addressed by exactly the kind of artefact this report represents. The audit's closing judgment is therefore the one it opened with, now earned by a hundred pages of evidence: this is a well-built seed with a small number of load-bearing omissions, every one of which has a known, sized, sequenced fix - and a team that reads this document and executes its first month will have converted the audit's severity counts into a moat.

Appendix A: Route Inventory

The complete URL surface of the application as implemented by the hash router at commit 86e2a66. All views are lazy-loaded except Home, which is statically imported into the shell chunk. The fallback route renders the NotFound view.

Route	View component	Function
#/	Home	Hero, search entry, featured listings, product navigation
#/properties	PropertiesView	Faceted browsing: area, type, purpose, beds, price, trust
#/properties/:id	PropertyDetailView	Gallery, trust breakdown, passport, mortgage calc, agent
#/compare	CompareView	Side-by-side comparison of saved listings
#/sell	ListPropertyView	Landlord listing submission wizard (14 labelled fields)
#/ask	AskKejaView	Trilingual AI advisor with escalation guard
#/deal-analyst	DealAnalystView	On-device document pre-screening
#/invest	InvestmentCalculatorView	Yield and affordability calculators
#/portfolio	InvestorDashboardView	Positions, metrics, alerts (recharts)
#/data	MarketDataView	Market statistics dashboards (recharts)
#/finance	FinanceView	Mortgage products and payment modelling
#/transact	TransactView	Transaction flow (presentational)
#/account	AccountView	Profile, sessions, preferences (auth-gated)
#/pro	ProWorkspaceView	Agent professional workspace
#/tokenize	TokenizeView	Tokenization trial: KYC, wallet, order book
#/manage	ManageView	Landlord units, tenants, ledger, tickets
#/tenant	TenantHubView	Tenant applications and status
#/diaspora	DiasporaHubView	Diaspora investor briefs
#/develop	DeveloperPortalView	Developer project listings
#/institutional	InstitutionalPortalView	Institutional investor surface
#/partners	PartnersView	Partner programme and applications
#/trust	TrustCenterView	Public claims register and honesty standard
#/ecosystem	EcosystemView	Nine-product overview
#/insights	InsightsView	Editorial articles index
#/insights/:slug	ArticleDetailView	Individual article

Route	View component	Function
#/areas/:slug	AreaGuideView	Neighbourhood guides (21 areas)
#/about	AboutView	Company story
#/contact	ContactView	Contact channels
#/legal	LegalView	Terms and policies
#/admin	AdminView	Moderation console (ungated - finding F-05)
#/valuation	ValuationDeskView	Valuation request surface
(fallback)	NotFoundView	404 experience

Table 40: Hash route table (32 routes plus fallback).

Appendix B: localStorage Registry

Every browser-storage key the application reads or writes, enumerated by domain. All keys use the *keja:* prefix (the tokenization blob and analytics buffer use versioned names). This registry is the complete inventory of platform state as of commit 86e2a66, and doubles as the migration checklist for the Phase 2 store twins described in Chapter 24.

Key	Contents	Owner
keja:session	Active session token, expiry, user snapshot	auth context
keja:users	Registered user records (client-side)	auth context
keja:pw	DJB2-hashed passwords (finding F-01)	auth context
keja:login-fails	Throttle counters per email	auth context
keja:favorites	Saved listing ids	core store
keja:leads	Captured lead stubs	core store
keja:profile	Local profile settings	core store
keja:chat-history	AI advisor conversations	core store
keja:language	UI language (en/sw/fr)	i18n
keja:recently-viewed	Recent listing trail	core store
keja:saved-searches	Search criteria + alerts	searchStore
keja:compare-list	Comparison tray members	searchStore
keja:notifications	Alert sweep results	searchStore
keja:audit	Admin audit trail (bounded 500)	adminStore
keja:submissions	Listing submissions queue	adminStore
keja:partners	Partner applications	adminStore
keja:feeds	Feed connection registry	adminStore
keja:settings	Platform settings	adminStore
keja:listing-reports	User reports (bounded 200)	adminStore
keja:user-listings	User-created listings	adminStore
keja:tenant	Rental applications incl. income data	tenantStore
keja:landlord	Units, tenants, payments, tickets	landlordStore
keja:valuations	Valuation requests	valuationStore
keja:pro	Professional workspace state	proStore
keja:diaspora	Diaspora briefs	diasporaStore
keja:dev-projects	Developer projects	devStore
investor-portfolio	Watchlists and positions	investorStore
keja-tokenize-v1	KYC records, wallet, ledger, orders (finding F-02)	tokenize context
keja.analytics.v1	200-event analytics ring	analytics
keja:listing-sigs	Listing signature cache (bounded 200)	autoListings

Table 41: localStorage key registry (28 keys).

Appendix C: Dependency Audit

The declared dependency list (69 dependencies, 12 dev dependencies) audited against actual imports in src/ at commit 86e2a66. The audit recommends deletion of every package in the unused list except where the Chapter 24 seam will adopt it deliberately (marked).

Package	Version	Disposition
zustand	5.0.6	Delete (re-evaluate at server-state seam)
next-auth	4.24.13	Delete (EOL line; do not revive for Phase 2)
@tanstack/react-query	5.82.0	Delete (adopt at seam if chosen)
@tanstack/react-table	8.21.3	Delete
zod	4.0.2	Keep (needed for F-19/F-20 boundary validation) or delete until then
@hookform/resolvers	3.10.0	Delete (serves only dead form.tsx)
react-hook-form	7.62.0	Delete (sole importer is dead ui/form.tsx)
@dnd-kit/core, sortable, utilities	6.x	Delete (3 packages)
next-intl	4.3.4	Delete (custom i18n in use)
uuid	11.1.0	Keep for the id helper (P1 craft item) or delete until adopted
date-fns	4.1.0	Delete
@reactuses/core	2.2.0	Delete
react-syntax-highlighter	15.6.6	Delete
@mdxeditor/editor	3.7.0	Delete
z-ai-web-dev-sdk	0.0.18	Delete (never imported; no key in bundle)
embla-carousel-react	8.6.0	Delete (dead ui/carousel.tsx)
react-day-picker	9.8.0	Delete (dead ui/calendar.tsx)
react-resizable-panels	3.0.0	Delete (dead layout primitives)
prisma + @prisma/client	6.19.2	Replace scaffold with authored schema (Ch. 24)
sonner	2.0.5	Verify: toast usage consolidates on it - audit found mixed toast paths

Table 42: Unused dependencies (zero imports in src/).

Package	Importers	Role
next	build	Framework, static export, metadata
react, react-dom	all views	Runtime
tailwindcss 4 + postcss	styles	Design tokens (CSS-first)
@radix-ui/* (subset)	18 ui files	Dialog/menu/tabs primitives in active use
lucide-react	44 files	Icon system
framer-motion	14 files	Entrance/stagger motion
recharts	6 files	Charts (dedup finding F-13)
class-variance-authority, clsx, tailwind-merge	ui layer	Class composition
next-themes	theme provider	Dark mode
@capacitor/core + splash + status	native shells	Android/iOS runtime

Table 43: Load-bearing dependencies in active use.

Appendix D: Document and Asset Inventory

The documentation suite (18 files) and notable asset classes tracked in the repository, with the audit's currency notes. The suite is unusually complete for a platform of this size and its quality materially assisted this audit; three items require freshness corrections noted in Chapter 26.

Document	Summary
README.md	Product, stack, deploy overview; nine-product table; doc index
docs/REPO_PICTURE.md	Engineering dossier (source of several audit baselines; live-URL claim needs update)
docs/DEPLOYMENT.md	Canonical Vercel runbook: architecture, CLI flow, DNS, quirks
docs/DOMAIN_CONFLICT_FIX.md	Superseded Netlify-era forensic record (historical)
docs/MOBILE.md	Capacitor build and sync runbook
docs/STRATEGY.md	Three-phase Kenya-first strategy with gates
docs/MARKETING_PLAYBOOK.md	Market snapshot, competitors, positioning, launch
docs/KENYA_PARTNER_PROPOSALS.md	Twenty named partner targets with tier model
docs/DATA_DICTIONARY.md	Field-level dataset documentation (auto-generated)
docs/REVIEW_ACTIONS.md	External review tracker (cites deleted test - update needed)
docs/pdf/ (6 files)	Rendered PDF editions of the primary documents
docs/cma/ (3 files)	CMA sandbox: testing plan, safeguards, tokenization quotation

Table 44: Documentation inventory.

Class	Count	Audit note
Property images (jpg+webp pairs)	84 subjects	WebP set unreferenced (~5 MB dead - F-23)
iOS splash screens	17	2.2 MB; complete device-class coverage
Icons and brand	6 + 6	Maskable variants generated deterministically
Screenshots	2	Manifest screenshot entries (narrow + wide)
SW, manifest, offline, robots	4	sitemap.xml missing from output (F-07)
Android resources	77 files	Adaptive icons, splash drawables, intent filters
iOS resources	26 files	Asset catalogs, AppDelegate, CapApp-SPM
Auto-Pilot scripts	8	Scanner, enrich, dedupe, quality, publish, commit, verify
Doc/PDF build scripts	18	keja-docs sources; committed artifacts

Table 45: Asset classes under public/ and native trees.

Appendix E: Glossary

Terms used in this report, defined as the audit uses them. Platform-specific coinages are marked; general terms are defined narrowly to their sense here.

Term	Definition
Ardhisasa	Kenya's national land information system; the reference point for genuine title verification claims
Auto-Pilot (platform)	The cron-driven pipeline that grows listing inventory through quality-gated commits to main
BFF / API seam	The single backend-for-frontend boundary this report recommends: typed API routes in front of persistence, auth, and payments
Claims register	The machine-readable inventory (src/data/claims.ts) of what the platform asserts about itself, surfaced in the Trust Center
CMA sandbox	Capital Markets Authority regulatory sandbox; the admission path contemplated for the tokenization ambitions
Daraja	Safaricom's developer platform for M-Pesa APIs (STK push, C2B, B2C)
DJB2	A 1997 non-cryptographic string hash; unsuitable for password storage (finding F-01)
DPA 2019 / ODPC	Kenya Data Protection Act 2019 and its regulator, the Office of the Data Protection Commissioner
Escalation guard	Engine behaviour routing legal, tax, and valuation questions to human handoff
FACT / ESTIMATE / ASSUMPTION / REPORTED	The AI engine's answer-labelling taxonomy underpinning the honesty system
Hash routing	URL navigation via the fragment (#/...) - offline-friendly, search-invisible
Hobby plan attribution	Vercel's requirement that Hobby deploys attribute to the team owner; workaround: commits authored as the owner
keja:* keys	The localStorage namespace for all platform state (28 keys; Appendix B)
localStorage	Browser-origin storage: ~5 MB, single-device, client-mutable - the platform's entire persistence layer today
Maskable icon	PWA icon variant with a safe zone that survives Android adaptive masking
PSP	Payment service provider; the licensed intermediary recommended for M-Pesa integration
Prerendering	Emitting static HTML per route at build time so crawlers and shares see real content
Property Passport	The per-listing identity block (KEJA-XXX) tying a listing to its evidence trail
RUM	Real-user monitoring of field performance (Core Web Vitals)
Store twin (platform)	A server-backed counterpart behind an existing domain-store hook signature, retaining localStorage as offline cache
STK push	M-Pesa online payment initiation that prompts the payer on their handset
Trust ceiling	Hard cap on computed trust scores, capping synthetic listings below verified ones
Vercel prebuilt flow	CLI sequence (pull, build, deploy --prebuilt) decoupling build from deploy

Term	Definition
WCAG AA	Web Content Accessibility Guidelines conformance level; 4.5:1 contrast threshold for body text
WhatsApp-first	The product's reliance on WhatsApp as the primary communication and conversion rail

Table 46: Glossary of terms.

Appendix F: Methodology Notes and Reproduction

Reproducing the audit

Every quantitative claim in this report is reproducible from the repository at commit 86e2a66 with the following commands. File counts: `git ls-files | wc -l`. Source inventory and line counts: `git ls-files 'src/**' | xargs wc -l`. Bundle composition: build with `NEXT_STATIC=1` and list `out/_next/static/chunks` with sizes, then cross-reference the script tags in `out/index.html` for first-load membership. Dead-code census: for each `src/components/ui/*.tsx`, `grep` its import path across `src/` excluding itself. Type-escape census: `grep` for `' : any'`, `'as any'`, `'@ts-ignore'`, `'@ts-expect-error'`, non-null assertion patterns in expressions. Deleted-test archaeology: `git show ae0b2e3^ --stat -- 'src/**/*.test.*'` and read the pre-rebuild `package.json` scripts. Secrets sweep: pattern `grep` across `git log -p` for token prefixes, plus `--diff-filter=A` on `.env` paths. LocalStorage census: `grep` for the storage accessor and key constants across `src/lib`.

Tooling and standards referenced

External references grounding the recommendations: Next.js documentation on rendering strategies (static generation ranked first for SEO); Safaricom Daraja developer documentation and 2026 integration guides (STK push flow, C2B callbacks, merchant provisioning timeline of roughly five weeks, escrow patterns via B2C and reversal); Kenya Data Protection Act 2019 as implemented by the Office of the Data Protection Commissioner (controller/processor registration, sensitive-data obligations); current testing-stack practice for React SPAs (Vitest with React Testing Library for unit and component layers, Playwright for end-to-end and PWA contexts); WCAG 2.1 AA for the contrast and motion findings; OWASP Top 10 (2021) for the security register categories. Market context drew on 2025-2026 coverage of the Kenyan proptech landscape including BuyRentKenya's position as the region's most-trafficked dedicated portal and the M-Pesa-integrated property-management trend.

Confidence and freshness

Code-level findings carry high confidence: each was verified directly against the repository state at the audited commit, and the audit's reproduction commands re-derive them. Bundle and asset measurements describe the build artifact sampled during the engagement and will drift as deploys continue; the findings they support (duplication, dead imagery) are structural and will persist until remediated. Regulatory and integration guidance reflects sources current as of September 2026 and should be re-validated with counsel and the relevant institutions before binding decisions. This report should be read alongside the repository's own claims register: where the two disagree, this report's evidence citations govern, and the disagreement itself is a documentation-freshness finding to close in the Chapter 26 craft items.